

廣州南方學院
NANFANG COLLEGE · GUANGZHOU

《企业项目实践》报告

项目名称：VANPINE SRM 全球供应商管理系统的设计与实现

第 X 组项目组成员与分工、成绩评定

学号	姓名	专业	主要工作	课程设计成绩
		23 计科	PM、组长	

		23 计科	系统分析	
		23 计科	系统设计	
		23 计科	系统编码	
		23 计科	系统测试	
平 均 成 绩				

2026 年 11 月 6 日

目 录

第 1 章 绪论.....1

1.1 课题背景与意义.....1

1.2 国内外研究现状.....2

1.3 课题研究的目标.....3

1.4 论文组织结构.....4

第 2 章 开发工具和技术简介.....5

2.1 Java 17、Spring Boot 3 与 Spring Cloud Alibaba.....5

2.2 Vue 3 与 Element Plus.....6

2.3 MySQL 8 与 MyBatis-Plus.....6

2.4 Redis、RabbitMQ 与 Nacos.....7

2.5 Python FastAPI、LangChain 与 Milvus.....7

第 3 章 系统分析.....9

3.1 可行性分析.....9

3.1.1 技术可行性.....9

3.1.2 经济可行性.....9

3.1.3 操作可行性、社会可行性等.....10

3.1.4 可行性分析结论.....10

3.2 外部接口需求.....10
3.2.1 用户界面需求.....11
3.2.2 硬件接口需求.....11
3.2.3 软件接口需求.....11
3.3 系统功能需求.....12
3.3.1 功能概述.....12
3.3.2 系统用例.....13
3.3.3 入驻审核与租户管理.....14
3.3.4 供应商画像与智能匹配.....16
3.3.5 询盘报价与比价授标.....17
3.3.6 合同与交付履约.....19
3.3.7 风险监控与 AI 助手.....20
3.4 系统非功能需求.....21
3.4.1 系统运行需求.....21
3.4.2 系统安全需求.....21
3.4.3 系统性能需求.....22
3.4.4 其他需求.....22
第 4 章 系统设计.....23
4.1 系统架构设计.....23

4.1.1 业务架构设计.....23

4.1.2 逻辑架构设计.....24

4.1.3 技术架构设计.....25

4.1.4 网络架构设计.....26

4.2 系统模块设计.....27

4.2.1 总体模块结构图.....27

4.2.2 平台治理子系统.....28

4.2.3 智能撮合子系统.....29

4.2.4 采购交易子系统.....30

4.2.5 交付履约子系统.....31

4.3 系统接口设计.....32

4.3.1 用户接口.....32

4.3.2 外部软硬件接口.....32

4.3.3 内部接口.....33

4.4 数据库设计.....34

4.4.1 数据需求分析.....34

4.4.2 概念结构设计.....38

4.4.3 逻辑结构设计.....42

4.4.4 物理结构设计.....47

4.5 非功能设计.....52

第 5 章 系统实现.....54

5.1 系统主界面.....54

5.2 入驻审核与租户管理的实现.....55

5.3 供应商画像与智能匹配的实现.....56

5.4 询盘报价与比价授标的实现.....58

5.5 交付履约与风险监控的实现.....59

5.6 AI 助手与知识库的实现.....60

第 6 章 系统测试.....62

6.1 测试方法.....62

6.2 测试用例.....62

6.3 测试结果.....64

第 7 章 系统特色和创新.....65

7.1 系统特色.....65

7.2 系统创新点.....66

第 8 章 总结.....67

参考文献.....69

附录.....70

附录 A　网关 JWT 鉴权与租户头注入.....70

附录 B　供应商智能匹配评分.....70

附录 C　用户表建表语句.....71

致谢.....72

第 1 章 绪论

本章介绍 VANPINE SRM 全球供应商管理系统的研究背景与意义，梳理国内外供应商管理与采购协同平台的研究与应用现状，明确本课题的研究目标，并给出论文的组织结构，为后续的系统分析、设计与实现提供依据。

1.1 课题背景与意义

近年来，全球供应链格局加速重构，原材料价格波动、地缘政治冲突、贸易合规要求趋严等因素叠加，使企业的采购活动从“买到货”转变为“买得稳、买得合规、买得可追溯”。与此同时，跨境电商与中小制造企业的国际化程度不断提高，企业需要在全球范围内寻找供应商，并对供应商的资质、产能、出口经验、财务状况和履约表现进行持续评估。然而，大量中小企业仍依赖展会、熟人推荐和 B2B 平台搜索寻找供应商，依靠 Excel 和电子邮件管理询价与报价，依靠人工追问跟踪交付进度，供应商信息分散、报价难以横向比较、合规风险事后才暴露等问题十分突出[1][2]。

在此背景下，开发一套面向跨境采购的供应商关系管理（Supplier Relationship Management，SRM）平台具有重要的现实意义。系统将供应商入驻、资质审核、企业画像、智能匹配、询盘报价、比价授标、电子合同、交付验收和风险监控串联为完整闭环，把分散在个人邮箱和聊天记录中的供应商信息沉淀为企业级资产；通过向量检索与大语言模型实现供需智能匹配并给出可解释的推荐理由，降低采购人员

的信息筛选成本；通过制裁名单筛查、尽职调查和交付风险预警，将合规与风险管理前置。对于课程实践而言，本课题综合运用软件工程、数据库、微服务、前端工程和人工智能等技术，能够较全面地锻炼需求分析、系统设计、编码实现与测试能力。

1.2 国内外研究现状

国外在供应商管理与采购数字化领域起步较早，已形成较为成熟的产品体系。SAP Ariba 提供从供应商寻源、合同管理到采购支付的端到端能力，其 Supplier Management 模块支持供应商注册、资质审核、绩效评估和风险监控[7]；Coupa 以业务支出管理为核心，强调采购申请、审批、供应商协同与费用分析的一体化[8]；Jaggaer、Ivalua 等厂商则在供应商准入、合规筛查和品类管理方面形成了差异化能力。近年来，Scoutbee 等创新企业开始将语义搜索与知识图谱用于供应商发现，尝试解决传统目录检索“关键词不匹配就找不到”的问题。总体来看，国外产品功能完整、生态成熟，但实施成本高、配置复杂，对中小企业的资金与信息化能力要求较高。

国内供应商管理平台的发展与产业互联网同步推进。用友 BIP、金蝶云星空等企业管理软件厂商在采购云中集成了供应商准入、询比价和合同管理能力；阿里巴巴国际站、1688 等 B2B 平台汇聚了大量供应商资源，并提供信用与交易保障；京东企业购、震坤行等平台则在工业品采购与履约服务方面形成了规模优势。近年来，

国内研究开始关注知识图谱、推荐算法和大语言模型在供应商推荐、采购文本处理与合同审查中的应用[4][5]。但现有平台多以信息展示和交易撮合为主，AI 能力多停留在客服问答或简单推荐层面，缺乏对供应商画像的深度结构化、对匹配结果的可解释输出，以及对跨境合规与交付风险的一体化治理。

综合国内外现状可以看出，面向跨境采购的一体化 SRM 平台仍有较大的创新空间：一是需要将 AI 匹配与业务规则结合，既保证召回率，又保证结果可解释；二是需要把制裁名单、股权关系、物流与天气等外部风险信号接入交易链路；三是需要以多租户方式服务不同企业，保证数据隔离与权限可控。本课题正是在这一背景下提出的。

1.3 课题研究的目标

本课题的总体目标是设计并实现一个面向跨境采购的 B2B 供应链协同平台，具体包括以下几个方面：

（1）功能目标：实现供应商入驻与审核、企业画像、智能匹配、尽职调查、询盘报价、比价授标、电子合同、交付验收、风险监控、AI 助手与平台治理等核心功能，形成从寻源到交付的业务闭环。

（2）技术目标：采用微服务架构与多租户模型，保证系统可扩展、可隔离；将 AI 计算能力独立部署，实现业务与 AI 解耦；通过向量检索与大模型生成提升匹配与分析的智能化水平。

（3）质量目标：核心接口响应时间控制在 2 秒以内，AI 分析接口控制在 8 秒以内；系统支持不少于 500 个并发用户，具备完善的鉴权、审计与降级机制。

（4）成果目标：完成可运行的系统原型、数据库设计、接口文档与测试用例，并撰写课程设计报告。

1.4 论文组织结构

全文共分八章。第 1 章为绪论，介绍课题背景、研究现状与研究目标；第 2 章介绍系统开发所使用的工具与技术；第 3 章进行系统分析，包括可行性分析、外部接口需求、功能需求与非功能需求；第 4 章进行系统设计，包括架构设计、模块设计、接口设计与数据库设计；第 5 章介绍系统主要功能的实现效果；第 6 章介绍系统测试方法与测试结果；第 7 章总结系统特色与创新点；第 8 章对全文进行总结并展望后续工作。

第 2 章 开发工具和技术简介

本章介绍系统开发过程中使用的主要工具与技术。系统采用前后端分离与微服务架构，后端基于 Java 17 与 Spring Cloud Alibaba 构建，前端基于 Vue 3 与 Element Plus，AI 能力由独立的 Python 服务提供，数据层使用 MySQL、Redis、RabbitMQ 与 Milvus。

2.1 Java 17、Spring Boot 3 与 Spring Cloud Alibaba

系统后端使用 Java 17 作为开发语言，基于 Spring Boot 3.3.7 构建业务服务。Spring Boot 通过自动装配、起步依赖和内嵌容器简化了企业级应用的搭建，使开发者可以专注于业务逻辑[12]。系统按照业务域拆分为平台、供应商、交易、交付和 AI 五个微服务，使用 Spring Cloud Alibaba 的 Nacos 实现服务注册与配置管理，使用 Spring Cloud Gateway 作为统一入口，通过 OpenFeign 完成服务间的同步调用，并通过 Spring Cloud LoadBalancer 实现客户端负载均衡。

在工程组织上，系统采用 Maven 多模块结构，将公共工具、服务契约、框架能力和业务服务分层管理：gsc-common 存放统一响应体、异常与枚举；gsc-api 存放 Feign 契约与跨服务 DTO；gsc-framework 提供安全、JWT、多租户、消息队列等公共能力；gsc-gateway 负责路由与鉴权；五个 gsc-service-* 模块分别承载具体业务。模块之间保持单向依赖，避免循环引用。

2.2 Vue 3 与 Element Plus

系统前端采用 Vue 3 的组合式 API 开发，使用 Vite 作为构建工具，Element Plus 作为 UI 组件库。Vue 3 的响应式系统与组件化机制适合构建复杂的后台管理界面；Vite 提供快速的冷启动与热更新，显著提升开发效率[13]。Element Plus 提供了表格、表单、对话框、树形控件等大量企业级组件，配合自定义主题可以快速搭建统一风格的界面。

前端使用 Pinia 管理认证、权限和全局状态，使用 Vue Router 实现动态路由与路由守卫，根据用户角色与权限码生成菜单；使用 Vue I18n 实现中英文切换，使用 Tailwind CSS 4 辅助布局。所有接口通过 Axios 统一封装，自动携带访问令牌，并在令牌过期时尝试刷新，保证会话的连续性。

2.3 MySQL 8 与 MyBatis-Plus

系统使用 MySQL 8.0 作为关系型数据库，采用 InnoDB 存储引擎与 utf8mb4 字符集，支持事务、行级锁与外键约束。数据库表统一使用小写下划线命名，主键采用雪花算法生成的 BIGINT，业务表统一包含 tenant_id、create_time、update_time 和 is_deleted 字段，以支持多租户隔离、审计与逻辑删除。

持久层使用 MyBatis-Plus 3.5.7。对于单表增删改查与条件查询，直接使用 BaseMapper 与 LambdaQueryWrapper；对于多表关联、聚合统计和复杂动态条件，则编写 XML 映射，显式列出查询字段，禁止使用 SELECT *。MyBatis-Plus 的分

页插件与自动填充机制减少了重复代码，使持久层保持简洁。

2.4 Redis、RabbitMQ 与 Nacos

Redis 用于缓存登录令牌、权限数据、验证码、分布式锁和 AI 会话上下文，利用其内存存储与过期机制提升访问速度并支撑多实例部署。RabbitMQ 用于服务间的异步通知，例如询盘发布、授标结果、合同确认和交付风险预警；系统通过消息确认、消费重试与死信队列保证消息可靠投递，并以业务键实现幂等消费。

Nacos 同时承担服务注册中心与配置中心职责。各服务启动时向 Nacos 注册实例，网关通过服务名进行负载均衡转发；公共配置与各服务配置集中存放在 Nacos 的命名空间中，本地配置文件只保留端口、服务名和兜底配置，便于多环境部署与统一运维。此外，系统使用 Micrometer Tracing 与 Zipkin 实现链路追踪，在日志中透传 traceId，便于定位跨服务调用问题。

2.5 Python FastAPI、LangChain 与 Milvus

AI 服务使用 Python 3.9+ 与 FastAPI 开发。FastAPI 基于 Starlette 与 Pydantic，具有异步处理、自动生成 OpenAPI 文档和数据校验能力，适合承载大模型调用与向量检索等 I/O 密集型任务[14]。AI 服务通过 HTTP 接口接收 Java 后端传入的供应商画像、采购需求和报价数据，完成向量化、语义召回、综合评分与文本生成。

向量化使用 OpenAI 兼容的 Embedding API（默认 BAAI/bge-m3，输出 1024

维向量），向量存储与检索使用 Milvus[6]。大语言模型通过 LangChain 的 OpenAI 适配器调用 DeepSeek，用于生成匹配理由、报价分析、授标理由、询盘填写建议和运输天气建议。AI 服务只负责计算与生成，不直接持有业务事务；当模型或向量库不可用时，业务主流程仍可继续，保证系统的可用性。

第 3 章 系统分析

系统分析是软件开发的关键阶段。本章从可行性、外部接口、功能需求和非功能需求四个方面对 VANPINE SRM 平台进行分析，明确系统“为谁做、做什么、做到什么程度”，为第 4 章的系统设计提供依据。

3.1 可行性分析

本节从技术、经济、操作与社会四个维度分析项目实施的可行性，并给出可行性结论。

3.1.1 技术可行性

系统采用的 Java、Spring Boot、Vue、MySQL、Redis、RabbitMQ、Nacos 等技术均为成熟的开源技术，社区活跃、文档完善，开发团队具备相应的开发经验。AI 部分使用 Embedding API、Milvus 与 DeepSeek，均提供标准的 HTTP 接口，接入成本较低。微服务架构虽然增加了服务治理的复杂度，但通过统一的父 POM、公共框架模块和 Nacos 配置中心可以有效降低维护成本。项目已在本地完成五个后端服务、前端和 AI 服务的联调，验证了技术方案的可行性。主要风险在于外部 API 的稳定性与调用成本，可通过超时控制、降级策略、缓存与调用量限制加以缓解。

3.1.2 经济可行性

系统全部采用开源技术栈，无需购买商业中间件授权；开发阶段可使用本地

Docker 环境与云服务器，硬件成本较低。主要变动成本来自 Embedding 与 DeepSeek 的 API 调用，系统通过只对 Top 10 候选生成理由、对画像文本做截断、对重复请求做缓存等方式控制调用量。系统上线后可以显著减少采购人员寻找供应商、整理报价和跟踪交付的时间，降低因供应商合规问题导致的交易风险，其带来的效率收益与风险收益远高于开发与运营成本。因此，项目在经济上是可行的。

3.1.3 操作可行性、社会可行性等

系统采用 B/S 架构，用户只需使用浏览器即可访问，无需安装客户端；界面按照角色动态生成菜单，采购商、供应商、平台管理员和尽调机构进入系统后只看到与自身职责相关的功能，降低学习成本。系统提供中英文界面，便于跨境用户使用。在社会与法律层面，系统对供应商进行制裁名单筛查并保留审计日志，有助于企业履行合规义务；同时，系统对个人信息与企业敏感数据进行权限隔离，符合数据安全保护的基本要求。

3.1.4 可行性分析结论

综合以上分析，系统在技术、经济、操作和社会环境方面均具备可行性，项目目标明确、技术路线成熟、成本可控，可以进入需求分析与系统设计阶段。

3.2 外部接口需求

外部接口需求描述系统与用户、硬件设备、第三方软件之间的交互方式。本节从用户界面、硬件接口和软件接口三个方面进行说明，具体接口设计见 4.3 节。

3.2.1 用户界面需求

系统面向企业用户，界面应简洁、专业、信息密度适中。整体采用左侧导航、顶部工具栏、右侧内容区的后台管理布局；主色调为深蓝与亮蓝，搭配浅灰背景，突出数据与操作按钮。列表页支持筛选、分页、排序与批量操作；表单页按业务分组，必填项明确标识，提交前进行校验；详情页以卡片和分组表单呈现企业画像、询盘、合同和交付信息。系统支持中英文切换，并根据角色动态生成菜单，避免用户看到无权限的功能。重要操作（授标、合同确认、验收）需要二次确认，AI 写操作必须由用户显式确认后执行。

3.2.2 硬件接口需求

系统为纯软件平台，不依赖专用硬件。客户端只需具备可运行现代浏览器的个人计算机或移动设备；服务器端需要部署应用服务器、数据库服务器和对象存储。若企业需要上传纸质资质或合同扫描件，可使用普通扫描仪或手机拍照后上传；若需要打印合同，可使用常规打印机。系统不直接对接身份证读卡器、支付终端等专用设备，相关能力通过第三方软件接口或人工录入实现。

3.2.3 软件接口需求

系统需要接入以下第三方软件接口：一是 Embedding API，用于将供应商画像和采购需求转换为向量；二是 Milvus 向量数据库，用于向量存储与相似度检索；三是 DeepSeek 大模型接口，用于生成匹配理由、报价分析和授标说明；四是天气

数据接口，用于查询航线与港口天气并生成运输建议；五是制裁名单数据源，用于定期同步和实时筛查；六是对象存储服务，用于保存资质材料、合同附件和交付凭证。所有外部接口均通过配置项注入地址与密钥，禁止在代码中硬编码。

3.3 系统功能需求

本节首先概述系统的整体功能，然后给出系统角色与顶层用例，并选择入驻审核、企业画像与智能匹配、询盘报价与比价授标、合同与交付履约四组核心用例进行详细描述。

3.3.1 功能概述

VANPINE SRM 是一个面向跨境采购的 B2B 供应链协同平台，服务对象包括采购企业、供应企业、第三方尽调机构和平台运营方。系统以“入驻审核—供应商画像—智能匹配—询盘报价—比价授标—电子合同—交付验收—风险监控”为主线，将采购全链路纳入统一平台。供应商入驻时结构化填写企业基本信息、供应品类、产能、认证资质、出口经验和财务履约记录，平台审核后开通租户并将画像向量化入库；采购方发布采购需求后，系统通过语义解析与向量召回获取候选供应商，再叠加品类、资质、产能、地域、行业、画像完整度和出口经验等维度进行加权评分，并生成可读的匹配理由。

在交易环节，采购方可一键创建询盘并邀请目标供应商，供应商在线提交报价，系统统一归集并进行 TCO（总拥有成本）拆解、历史价格对标和异常报价识别，

AI 给出排名、风险点与授标建议；授标后自动生成授标理由并通知相关供应商，随后进入电子合同签署与交付履约阶段。交付环节覆盖发货、物流跟踪、收货确认和到货验收，关键节点自动存证，系统结合天气、航线和历史履约数据识别延期风险并触发分级预警，必要时生成备选供应商、替代航线等应急处置建议。平台治理方面提供多租户管理、用户/角色/菜单权限、审计日志、消息通知、文件服务和制裁名单管理；AI 助手基于企业知识库提供 RAG 问答，并可在人工确认后调用业务接口完成写操作。

3.3.2 系统用例

系统采用多租户 + RBAC 权限模型，共包含七类角色：采购商（BUYER/BUYER_ADMIN）、供应商（SUPPLIER/SUPPLIER_ADMIN）、双重身份用户、企业管理员（TENANT_ADMIN）、超级管理员（SUPER_ADMIN）、尽调专员（DUE_DILIGENCE）和尽调事务所（FIRM）。不同角色通过菜单权限与数据权限访问不同功能。系统顶层用例如图 3-1 所示。

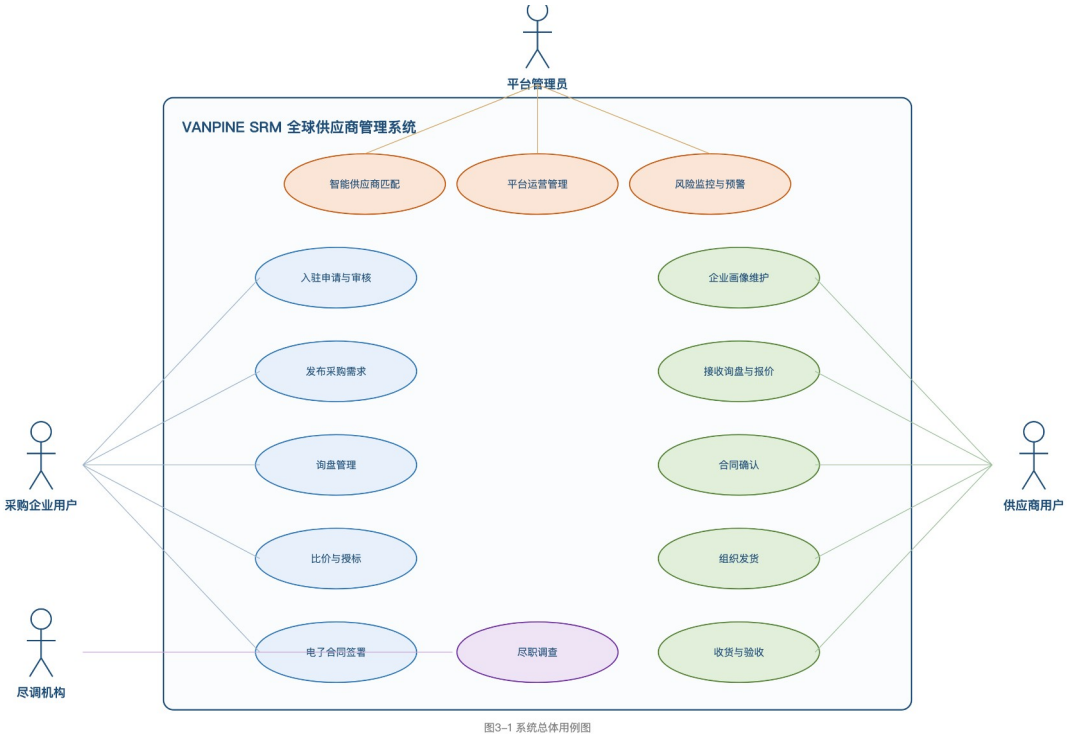


图 3-1　系统总体用例图

图 3-1 中，采购企业用户主要参与需求发布、询盘、比价、授标、合同签署和验收；供应商用户主要参与企业画像维护、接收询盘、提交报价、合同确认和发货；平台管理员负责入驻审核、平台运营、智能匹配规则维护和风险监控；尽调机构负责接收尽调任务并提交尽调报告。下面选择四组核心用例进行细化。

3.3.3 入驻审核与租户管理

入驻审核与租户管理是平台准入的第一道关口。申请企业提交入驻申请并上传营业执照、认证证书等材料，平台先进行制裁名单筛查，再由管理员审核；审核通过后系统创建租户、初始化管理员账号并分配默认角色与菜单。其子用例如图 3-2 所示。

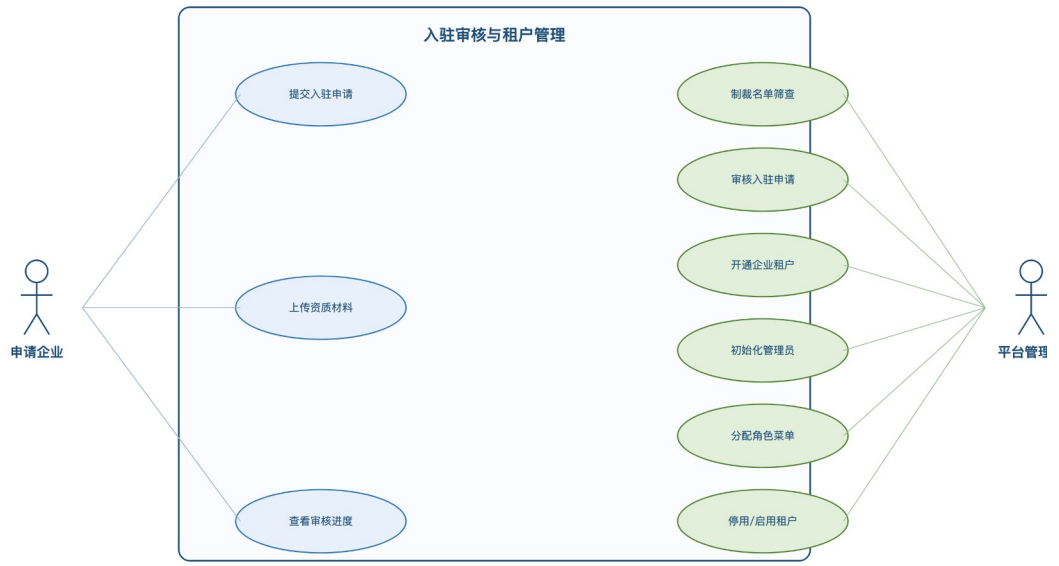


图3-2 入驻审核与租户管理子用例图

图 3-2　入驻审核与租户管理子用例图

用例编号	UC-01	**用例名称**	入驻审核
活动者（Actor）	平台管理员	**优先级**	高
描述	平台管理员审核企业提交的入驻申请，决定是否为其开通租户。		
输入	申请编号、审核结论、审核意见；企业基本信息、资质材料、制裁筛查结果。		
输出	审核结果、租户编号（审核通过时）、站内通知。		
前置条件	申请已提交，状态为“待审核”，且申请材料完整。		
基本流程	1. 管理员进入入驻预审列表，按状态或企业名称筛选；2. 打开申请详情，查看企业基本信息、资质附件与制裁筛查结果；3. 选择“审核通过”或“驳回”，填写审核意见；4. 审核通过时创建租户、初始化管理员账号与默认角色；5. 写入审计日志并通知申请人。		
可选流程	材料不完整时可退回补充；命中制裁名单疑似记录时转尽职调查。		
异常流程	租户编码已存在时提示重新生成；创建租户事务失败时整体回滚并提示重试。		

表 3-1　入驻审核用例表

用例编号	UC-02	**用例名称**	租户管理
活动者（Actor）	超级管理员	**优先级**	中
描述	超级管理员查看、停用、启用租户并维护订阅到期时间。		
输入	租户编号、操作类型、订阅到期时间。		
输出	租户状态、操作结果。		

前置条件	当前用户具备平台级租户管理权限。
基本流程	1. 进入租户管理列表；2. 按企业名称、状态、到期时间筛选；3. 选择目标租户执行停用或启用；4. 修改订阅到期时间；5. 记录操作日志。
可选流程	可批量导出到期租户清单，提前提醒续费。
异常流程	停用租户时该校验是否存在在途合同；存在时提示先处理在途业务。

表 3-2　租户管理用例表

3.3.4 供应商画像与智能匹配

供应商画像是智能匹配的基础。供应企业按基本信息、供应能力、采购能力、资质认证和履约表现五个部分维护画像，系统根据字段完整度计算画像完整度，并将画像文本向量化写入 Milvus。采购方发布需求后，系统解析需求文本、召回候选供应商并评分排序，最后生成可读的匹配理由。其子用例如图 3-3 所示。

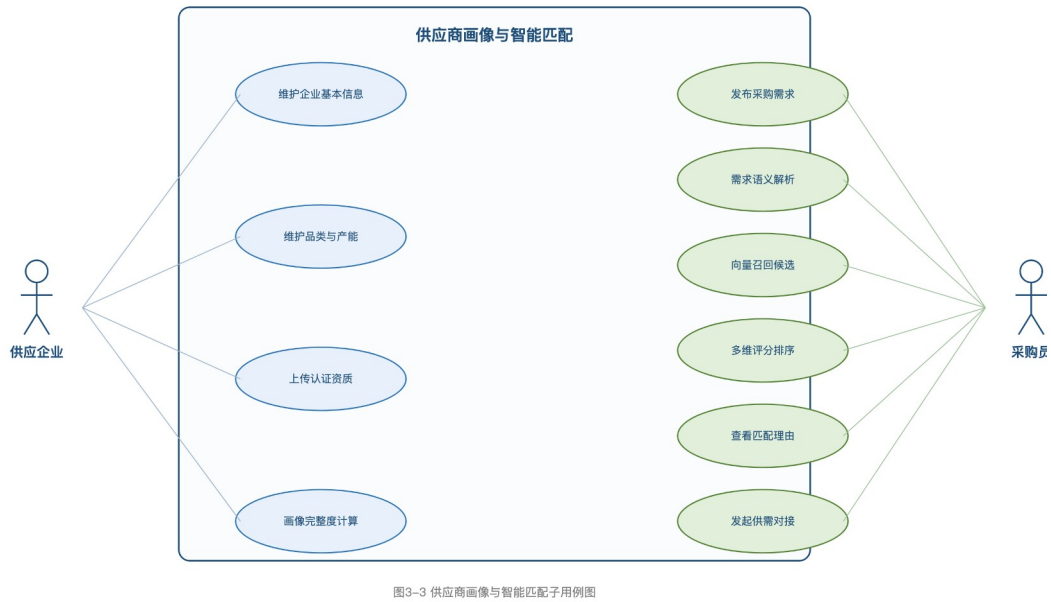


图 3-3　供应商画像与智能匹配子用例图

用例编号	UC-03	**用例名称**	企业画像维护
活动者（Actor）	供应商	**优先级**	高
描述	供应商维护企业基本信息、供应能力、资质与履约数据，形成结构化画像。		

输入	企业名称、国家、注册号、供应品类、年产能、认证资质、出口经验、主要产品、主要客户等。
输出	画像保存结果、画像完整度、向量化状态。
前置条件	用户已登录且属于供应企业。
基本流程	1. 进入企业画像页面；2. 分模块填写或修改画像字段；3. 上传营业执照、认证证书等附件；4. 保存画像，系统计算完整度；5. 画像变更后异步触发向量化更新。
可选流程	支持从入驻申请数据一键带入，减少重复填写。
异常流程	附件格式或大小不符合要求时提示；向量化失败时记录任务状态，由定时任务补偿。

表 3-3　企业画像维护用例表

用例编号	UC-04	**用例名称**	智能供应商匹配
活动者（Actor）	采购员	**优先级**	高
描述	根据采购需求从供应商库中召回候选并给出带理由的排序结果。		
输入	采购需求（品类、规格、数量、地域、资质要求、交期、描述）。		
输出	候选供应商列表、综合评分、匹配理由、风险提示。		
前置条件	采购需求已保存，供应商画像库中存在可用数据。		
基本流程	1. 采购员在需求详情页点击“智能匹配”；2. 系统解析需求并生成向量；3. Milvus 召回 Top 100 候选；4. 合并规则候选并按品类、资质等条件硬过滤；5. 七维加权评分并排序；6. 为 Top 10 生成匹配理由；7. 返回前端展示。		
可选流程	候选不足时可扩大品类或地域范围重新召回。		
异常流程	Embedding 或 Milvus 不可用时降级为基于规则的匹配；大模型超时时返回模板理由。		

表 3-4　智能供应商匹配用例表

3.3.5 询盘报价与比价授标

采购员可在需求基础上创建询盘，选择目标供应商并设置报价截止时间；供应商收到通知后在线报价，系统统一归集报价并进行 TCO 分析、异常识别和授标建议。其子用例如图 3-4 所示。

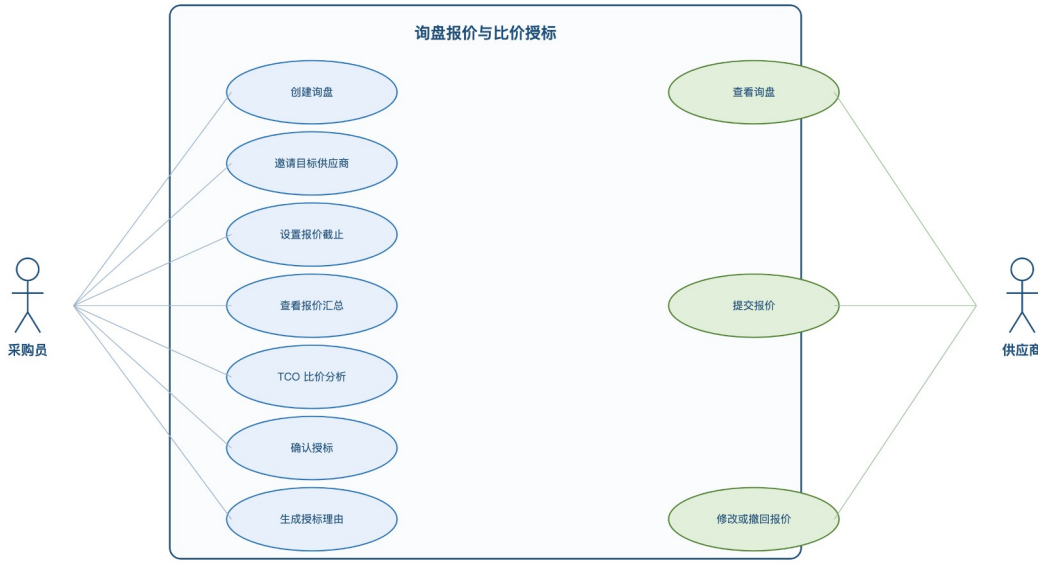


图3–4 询盘报价与比价授标子用例图

图 3-4　询盘报价与比价授标子用例图

用例编号	UC-05	**用例名称**	发布询盘
活动者（Actor）	采购员	**优先级**	高
描述	采购员基于采购需求创建询盘，邀请目标供应商参与报价。		
输入	需求编号、询盘标题、品类、数量、币种、交货地点、交货日期、报价截止时间、付款条件、邀请供应商列表。		
输出	询盘编号、邀请记录、站内通知。		
前置条件	需求存在且状态允许发起询盘；选择的供应商已通过认证。		
基本流程	1. 在需求详情页点击“发起询盘”；2. 系统带入需求字段，采购员补充商务条款；3. 选择目标供应商并确认；4. 保存询盘并生成邀请记录；5. 通过 RabbitMQ 发送询盘通知。		
可选流程	可保存为草稿后再次编辑；可追加邀请供应商。		
异常流程	报价截止时间早于当前时间时拒绝保存；供应商已被邀请时去重。		

表 3-5　发布询盘用例表

用例编号	UC-06	**用例名称**	比价与授标
活动者（Actor）	采购员	**优先级**	高
描述	对同一询盘下的多份报价进行横向比较，选择中标供应商并生成授标理由。		
输入	询盘编号、报价明细、比价维度（价格、交期、付款条件、运费、关税等）。		
输出	TCO 对比结果、AI 排名与风险提示、授标记录、授标理由。		

前置条件	询盘处于报价中或待授标状态，且至少有一份有效报价。
基本流程	1. 进入比价页面，查看报价汇总；2. 系统计算 TCO 成本并识别异常报价；3. 调用 AI 服务生成排名、风险点与授标建议；4. 采购员确认授标对象与数量；5. 保存授标记录并调用 AI 生成授标理由；6. 通知中标与未中标供应商。
可选流程	可指定部分数量授标给多家供应商；可暂存比价结果后再次进入。
异常流程	报价已撤回或超过有效期时禁止授标；授标数量超过可供数量时提示。

表 3-6　比价与授标用例表

3.3.6 合同与交付履约

授标完成后，采购员创建电子合同并上传签署件，供应商确认合同后进入交付阶段。交付覆盖发货、物流、收货、验收和风险处置，关键节点自动存证。其子用例如图 3-5 所示。

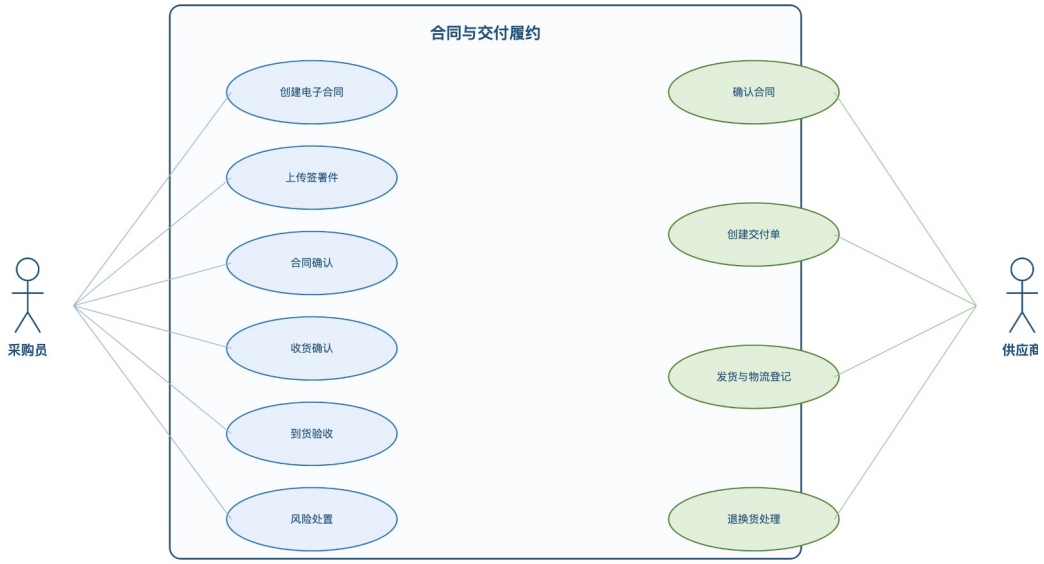


图3–5 合同与交付履约子用例图

图 3-5　合同与交付履约子用例图

用例编号	UC-07	**用例名称**	合同签署
活动者（Actor）	采购员、供应商	**优先级**	高
描述	采购员创建合同并上传签署件，供应商确认后合同生效。		
输入	询盘编号、中标报价编号、合同金额、数量、交期、付款条件、交付条款、合同		

	附件。
输出	合同编号、合同状态、签署记录、通知消息。
前置条件	询盘已完成授标，且中标报价有效。
基本流程	1. 采购员基于授标结果创建合同；2. 填写商务条款并上传签署件；3. 提交合同给供应商确认；4. 供应商查看合同并确认或拒绝；5. 确认后合同生效，进入交付计划。
可选流程	供应商可提出条款异议并退回修改；可上传补充协议。
异常流程	合同金额与授标金额不一致时提示确认；附件上传失败时保留草稿。

表 3-7　合同签署用例表

用例编号	UC-08	**用例名称**	交付验收
活动者（Actor）	采购员、供应商	**优先级**	高
描述	供应商发货并登记物流信息，采购员收货确认和到货验收。		
输入	交付单编号、发货数量、物流公司、物流单号、发货日期、收货数量、验收结果、不合格数量。		
输出	交付状态、验收结果、风险预警、违约罚金计算结果。		
前置条件	合同已生效，交付单处于待发货状态。		
基本流程	1. 供应商创建交付单并登记发货信息；2. 系统记录物流事件；3. 采购员确认收货；4. 采购员执行到货验收，录入合格数量与不合格数量；5. 系统更新交付状态并触发风险扫描。		
可选流程	支持分批交付与部分验收；不合格时可发起退换货流程。		
异常流程	收货数量与发货数量不一致时提示填写差异说明；验收超期时自动预警。		

表 3-8　交付验收用例表

3.3.7 风险监控与 AI 助手

风险监控模块聚合在途订单、物流轨迹、天气和供应商履约数据，将风险事件关联到具体供应商与订单，按风险分值触发分级预警，并支持生成备选供应商、替代航线和调拨方案等应急处置建议。AI 助手基于企业知识库提供 RAG 问答，可回答采购制度、合同条款和供应商履约情况等问题；当用户要求执行写操作（如创

建授标、修改交期、关闭询盘）时，AI 只能生成待确认提案，必须由用户点击确认后才会调用业务接口并记录审计日志。

3.4 系统非功能需求

非功能需求决定系统能否稳定、安全、高效地运行。本节从运行环境、安全、性能和其他质量属性四个方面提出要求。

3.4.1 系统运行需求

服务端建议配置 8 核 CPU、16 GB 内存、200 GB 磁盘的 Linux 服务器，分别部署应用服务、MySQL、Redis、RabbitMQ、Nacos 和 Milvus；AI 服务可单独部署，建议配备 GPU 或使用云端 Embedding 与推理服务。客户端要求 Chrome 90、Edge 90、Firefox 90 或 Safari 14 及以上版本，屏幕分辨率不低于 1366×768，网络带宽不低于 2 Mbps。系统支持 Docker Compose 一键部署开发环境，生产环境可使用 systemd 或容器编排管理服务进程。

3.4.2 系统安全需求

系统安全需求包括：所有接口统一经过网关鉴权，JWT 令牌包含用户、租户与角色信息，令牌过期后通过刷新令牌续期；密码使用 BCrypt 加密存储，禁止明文传输与记录；服务层通过租户拦截器与 SQL 条件双重保证数据隔离，防止跨租户访问；内部接口通过共享密钥与来源校验，禁止外部直接访问；关键操作记录审计日志，包含操作人、租户、目标对象、请求参数、IP 与耗时；AI 写操作必须二

次确认，模型不能直接修改业务数据；文件上传校验类型与大小，敏感配置通过环境变量或配置中心注入。

3.4.3 系统性能需求

系统应满足以下性能指标：普通列表与详情接口在正常负载下平均响应时间不超过 1 秒，95% 请求不超过 2 秒；登录、权限校验等高频接口在 Redis 缓存命中时不超过 300 毫秒；AI 匹配与比价分析接口在外部模型正常时不超过 8 秒，超时后返回降级结果；系统支持不少于 500 个并发在线用户，核心业务接口的吞吐量不低于 200 TPS；数据库慢查询比例低于 1%，关键查询命中索引；系统年可用性不低于 99.5%。

3.4.4 其他需求

系统应具备良好的可维护性与可扩展性：后端按业务域拆分服务，新增业务时优先在对应域内扩展，避免跨域耦合；接口统一返回 R<T> 结构，使用 Knife4j 生成在线文档；前端组件化开发，公共组件与业务页面分离；代码遵循统一的命名、注释与分层规范。系统还应提供部署文档、接口文档、数据库设计文档和用户操作手册，便于后续维护与移交。

第 4 章 系统设计

本章在第 3 章需求分析的基础上，对系统进行总体设计。首先介绍业务架构、逻辑架构、技术架构和网络架构，然后进行模块划分与核心流程设计，接着给出接口设计与数据库设计，最后说明非功能设计。

4.1 系统架构设计

系统采用前后端分离与微服务架构，按照“高内聚、低耦合”的原则划分服务边界，并通过网关统一对外暴露接口。本节从业务、逻辑、技术和网络四个视角描述系统架构。

4.1.1 业务架构设计

系统的业务架构围绕采购方、供应方和平台运营方三类主体展开，覆盖从需求发布到交付验收的完整链路。业务架构如图 4-1 所示。

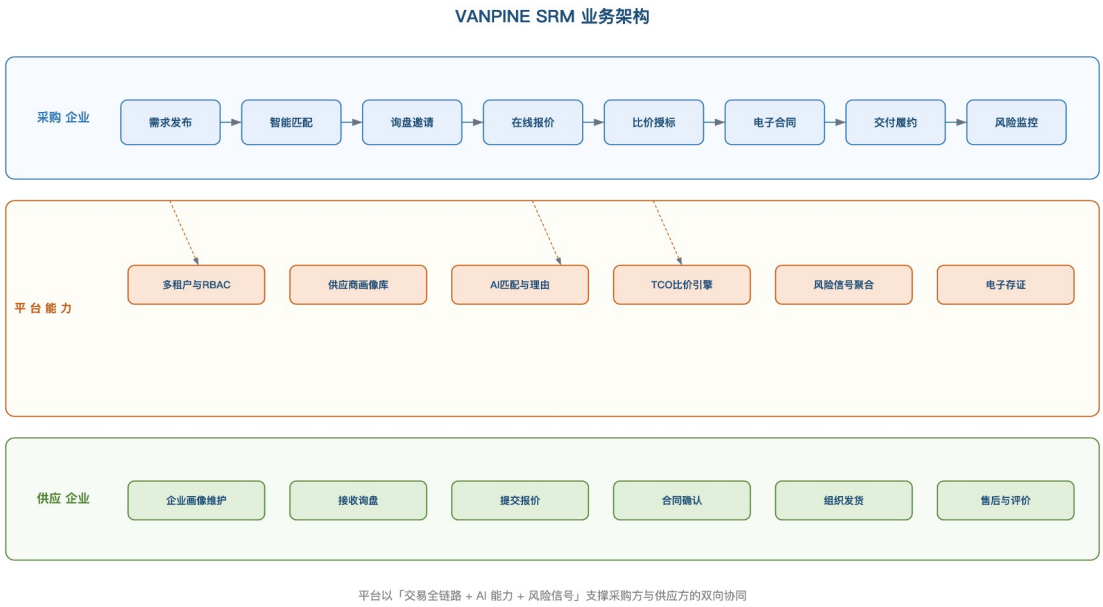


图 4-1　系统业务架构图

在业务架构中，采购企业负责发布需求、发起询盘、比价授标、签署合同和验收；供应企业负责维护画像、接收询盘、提交报价、确认合同和组织发货；平台方提供多租户与 RBAC、供应商画像库、AI 匹配与理由生成、TCO 比价引擎、风险信号聚合和电子存证等公共能力，支撑双方协同。平台能力与业务链路解耦，便于后续扩展新的品类、区域或服务模式。

4.1.2 逻辑架构设计

系统逻辑架构采用分层设计，自上而下分为用户访问层、接入层、业务服务层、能力支撑层、数据存储层和外部服务层，如图 4-2 所示。

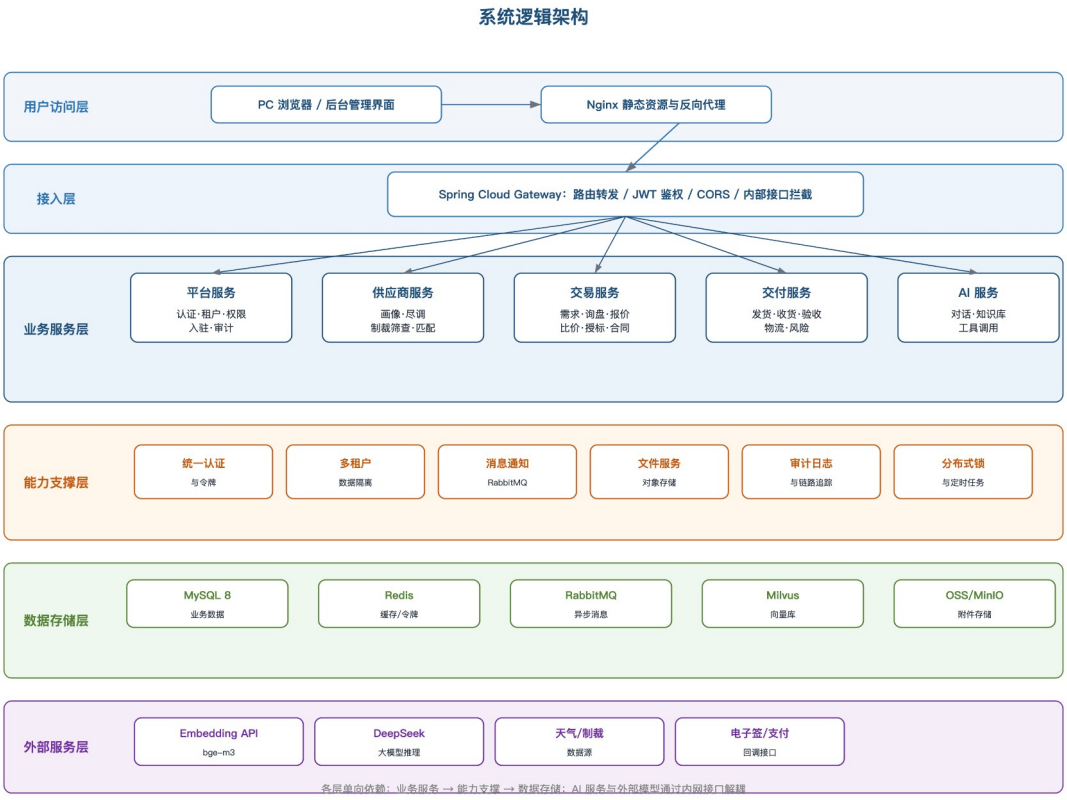


图 4-2　系统逻辑架构图

用户访问层由浏览器与后台管理界面组成；接入层使用 Spring Cloud Gateway

完成路由转发、JWT 鉴权、CORS 和内部接口拦截；业务服务层按业务域拆分为平台服务、供应商服务、交易服务、交付服务和 AI 服务，各服务通过 OpenFeign 同步调用、通过 RabbitMQ 异步通信；能力支撑层提供统一认证、多租户隔离、消息通知、文件服务、审计日志和分布式锁等公共能力；数据存储层包括 MySQL、Redis、RabbitMQ、Milvus 和对象存储；外部服务层包括 Embedding API、DeepSeek、天气与制裁数据源以及电子签/支付回调接口。各层之间保持单向依赖，业务服务不直接访问其他服务的数据表。

4.1.3 技术架构设计

技术架构是逻辑架构的物理实现与具体技术选型。系统技术架构如图 4-3 所示。

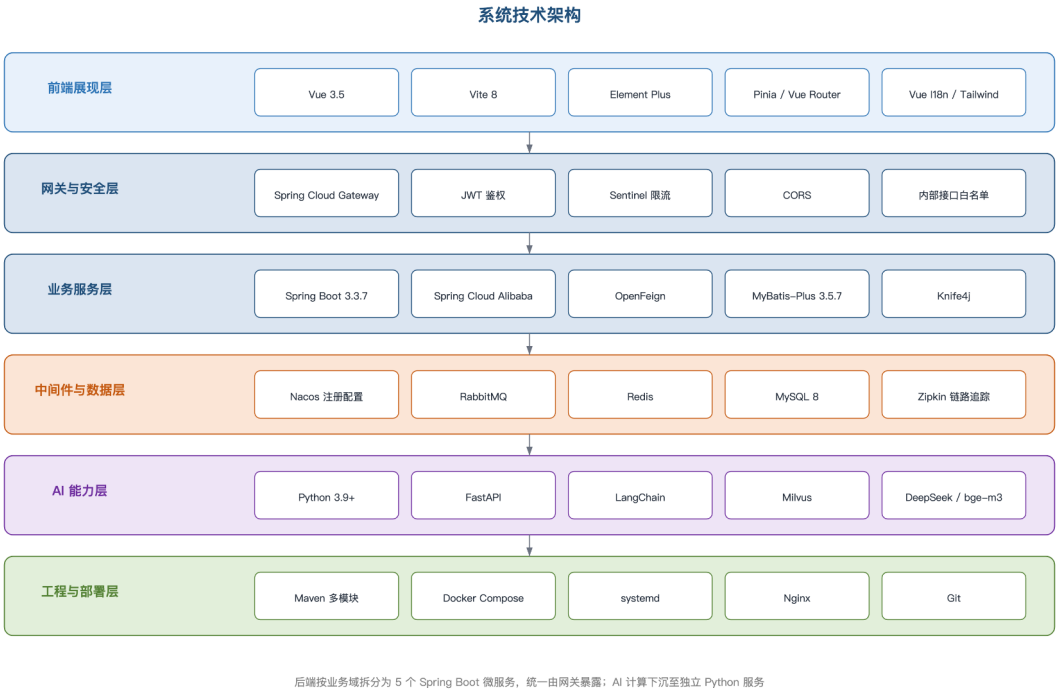


图 4-3　系统技术架构图

前端使用 Vue 3.5、Vite 8、Element Plus、Pinia、Vue Router、Vue I18n 和 Tailwind CSS 4；网关与安全层使用 Spring Cloud Gateway、JWT、Sentinel 和 CORS；业务服务层基于 Spring Boot 3.3.7、Spring Cloud Alibaba、OpenFeign、MyBatis-Plus 3.5.7 和 Knife4j；中间件与数据层使用 Nacos、RabbitMQ、Redis、MySQL 8 和 Zipkin；AI 能力层使用 Python 3.9+、FastAPI、LangChain、Milvus 以及 DeepSeek 与 bge-m3；工程与部署层使用 Maven 多模块、Docker Compose、systemd、Nginx 和 Git。

4.1.4 网络架构设计

系统部署架构按照公网访问区、应用服务区、数据与中间件区和外部服务区进行划分，如图 4-4 所示。

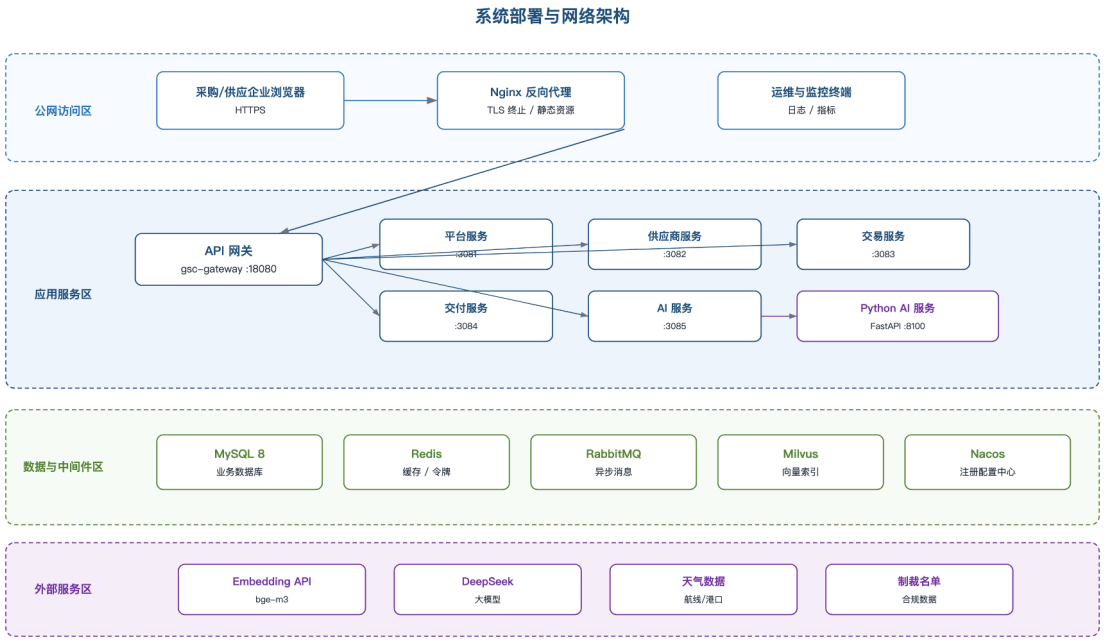


图 4-4　系统部署与网络架构图

用户通过 HTTPS 访问 Nginx，Nginx 负责 TLS 终止与静态资源分发，并将 API 请求反向代理到 API 网关；网关根据路由规则将请求转发到五个业务服务；业务服务统一访问 MySQL、Redis、RabbitMQ、Milvus 和 Nacos；AI 服务通过内网 HTTP 调用 Python AI 服务，后者再访问 Embedding API、DeepSeek、天气与制裁数据源。生产环境中，数据库与中间件部署在内网，外部服务通过防火墙白名单访问，网关与业务服务不直接暴露到公网。

4.2 系统模块设计

根据业务需求与模块化原则，系统划分为平台治理、智能撮合、采购交易、交付履约和 AI 智能服务五个子系统。本节给出总体模块结构，并对核心子系统进行详细设计。

4.2.1 总体模块结构图

系统总体模块结构如图 4-5 所示。每个子系统内部按照功能点进一步划分，模块之间通过接口或消息进行协作。



图4-5 系统总体模块结构图

图 4-5　系统总体模块结构图

4.2.2 平台治理子系统

平台治理子系统是系统的基础支撑，包含身份与访问、租户管理、用户管理、权限管理和平台治理五个模块，其结构如图 4-6 所示。



图4-6 平台治理子系统模块结构图

图 4-6　平台治理子系统模块结构图

身份与访问模块负责登录、JWT 令牌签发与校验、密码找回和 AI 护照；租户管理模块负责入驻申请、资质审核、租户开通和订阅到期管理；用户管理模块负责用户列表、创建与停用、个人信息和密码修改；权限管理模块负责角色定义、菜

单授权、按钮权限和数据权限；平台治理模块负责制裁名单、审计日志、消息通知和文件服务。平台治理子系统通过 gsc-api 中的 Feign 契约向其他服务提供租户、用户与权限信息，其他服务不直接访问其数据库。

4.2.3 智能撮合子系统

智能撮合子系统负责供应商画像、尽职调查、制裁筛查、采购需求、智能匹配和供需对接。核心的供应商匹配流程如图 4-7 所示。



图 4-7　供应商智能匹配活动图

匹配流程分为召回、过滤、评分和解释四个阶段。首先对采购需求进行文本清洗与语义解析，调用 Embedding API 生成需求向量；随后在 Milvus 中召回 Top 100 相似供应商，并与基于品类、地区的规则候选合并；若候选为空，则扩大品类或地域范围重新召回；接着过滤命中制裁名单或黑名单的企业；然后按品类覆盖率 30%、资质匹配率 25%、产品关键词 15%、地域 10%、行业 5%、画像完整度 10%、出口经验 5% 的权重计算综合得分；最后为得分最高的 10 家供应商生成匹

配理由，按得分排序返回前端。

4.2.4 采购交易子系统

采购交易子系统负责采购需求、询盘、询盘邀请、报价、比价分析与授标、合同管理。询盘到授标的核心时序如图 4-8 所示。

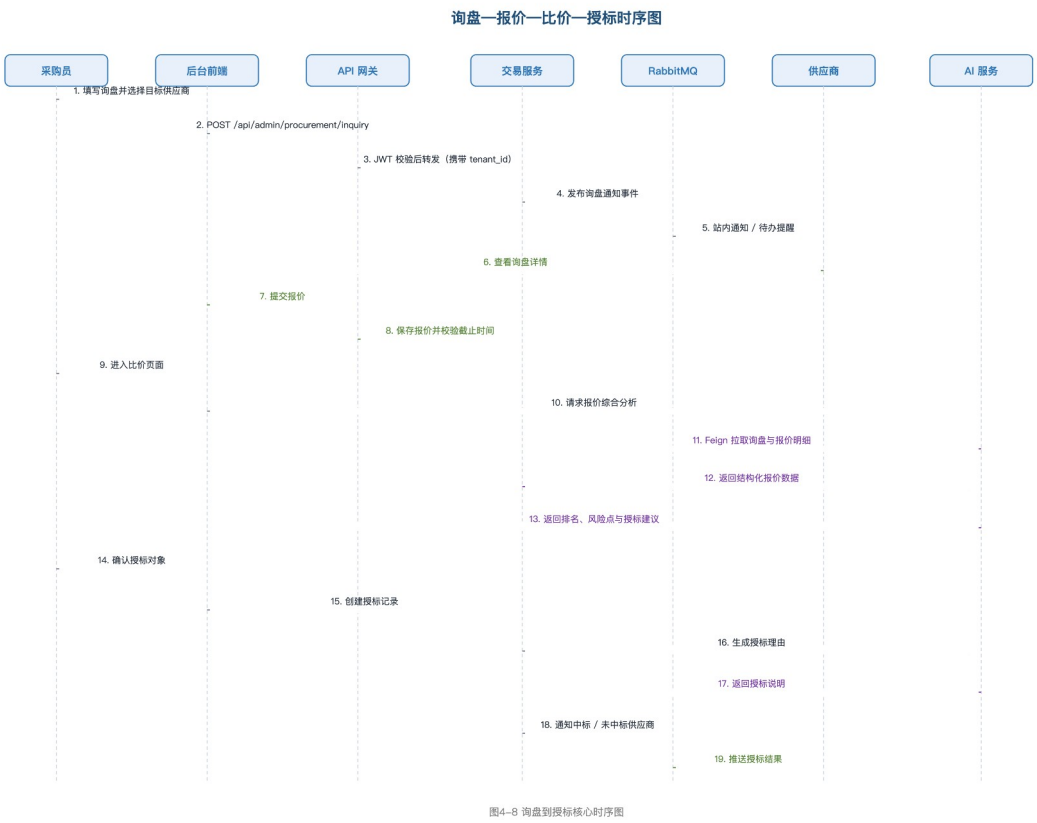


图 4-8　询盘到授标核心时序图

采购员在前端创建询盘后，请求经网关校验 JWT 并注入租户头后转发到交易服务；交易服务保存询盘与邀请记录，并通过 RabbitMQ 向供应商发送通知。供应商提交报价后，交易服务校验报价截止时间并保存报价。采购员进入比价页面时，前端调用 AI 服务，AI 服务通过 Feign 从交易服务拉取询盘与报价明细，完成

TCO 拆解、异常识别与排名，并返回风险点与授标建议。采购员确认授标后，交易服务创建授标记录、调用 AI 生成授标理由，并通过消息通知中标与未中标供应商。

4.2.5 交付履约子系统

交付履约子系统负责发货、收货、验收、物流跟踪、风险扫描和应急处置。交付履约活动如图 4-9 所示。

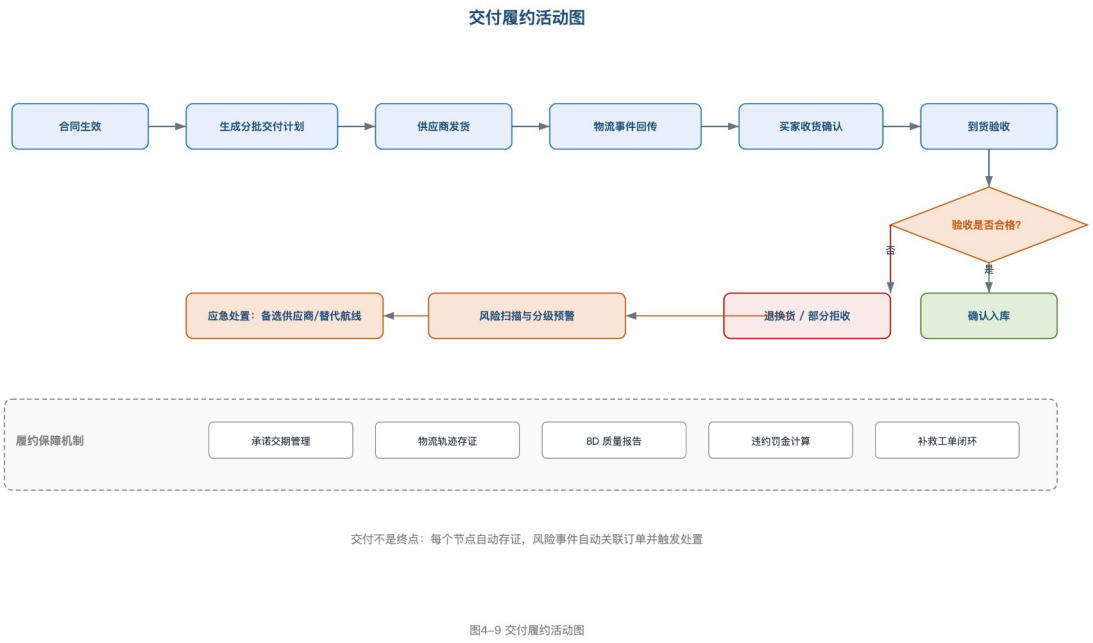


图 4-9　交付履约活动图

合同生效后，系统按合同约定生成分批交付计划；供应商发货并登记物流信息，物流事件持续回传；采购方确认收货并执行到货验收。若验收合格，则确认入库；若不合格，则进入退换货或部分拒收流程。系统同步执行风险扫描，将物流延误、天气异常、供应商履约偏差等信号转化为风险分值，按等级触发预警，并生成备选

供应商、替代航线或调拨方案等应急处置建议。交付单、物流事件、验收结果和风险处置过程均自动存证，形成可追溯的履约档案。

4.3 系统接口设计

系统接口分为用户接口、外部软硬件接口和内部接口三类。用户接口面向浏览器；外部接口面向第三方服务；内部接口用于微服务之间的协作。

4.3.1 用户接口

用户接口采用 B/S 架构，前端通过 Axios 调用后端 REST API。所有接口统一使用 /api 前缀，返回 R<T> 结构，包含 code、message 和 data 三个字段。前端按照业务模块封装 API 文件，例如 auth.js、procurement.js、delivery.js、match.js 和 ai.js；请求拦截器自动携带 Authorization 头，响应拦截器统一处理 401、403 和业务错误。页面采用 Element Plus 组件构建，表格分页、表单校验、弹窗确认和消息提示保持一致的交互规范。

4.3.2 外部软硬件接口

系统主要的外部软件接口如下：

（1）接口名称：Embedding 向量化接口。方法：HTTP POST。内容简介：将供应商画像或采购需求文本转换为 1024 维向量。输入参数：tenantId、text 或 requirementId、标题、品类、描述。返回结果：向量数组或写入状态。接口调用要求：需要 API Key，单次文本长度不超过 8000 字符，失败时重试 2 次并记录日

志。

（2）接口名称：DeepSeek 文本生成接口。方法：HTTP POST（OpenAI 兼容）。内容简介：生成匹配理由、报价分析、授标理由、询盘建议和天气处置建议。输入参数：模型名称、消息列表、温度、最大 token 数。返回结果：生成文本。接口调用要求：设置 60 秒超时，失败时降级为模板理由或返回空建议。

（3）接口名称：Milvus 向量检索接口。方法：gRPC/HTTP。内容简介：按租户写入供应商画像向量，按相似度召回候选。输入参数：collection、向量、topK、过滤条件。返回结果：候选 ID、距离与元数据。接口调用要求：集合按租户分区，查询必须带 tenant_id 过滤，向量库异常时降级为规则匹配。

（4）接口名称：天气数据接口。方法：HTTP GET。内容简介：查询航线、港口或目的地的天气与预警信息。输入参数：地点、日期范围。返回结果：天气状况、温度、风力、预警等级。接口调用要求：结果缓存 30 分钟，接口失败时不影响交付主流程，仅在风险提示中标注“天气数据暂不可用”。

4.3.3 内部接口

内部接口用于微服务之间的数据查询与状态回写，统一放在 gsc-api 模块中定义 Feign 契约，提供方使用 InternalXxxController 实现。内部接口路径统一以 /internal 开头，只允许携带内部密钥的请求访问，主要接口如表 4-9 所示。

表 4-9 主要内部接口清单

接口名称	提供方	方法	路径	说明
ApplicationClient	平台服务	GET	/internal/applications/{id}	查询入驻申请信息
ProfileClient	供应商服务	GET	/internal/profiles/{tenantId}	查询企业画像
RequirementClient	交易服务	GET	/internal/requirements/{id}	查询采购需求
InquiryClient	交易服务	GET	/internal/inquiries/{id}	查询询盘及报价
ContractClient	交易服务	GET	/internal/contracts/{id}	查询合同信息
DeliveryClient	交付服务	GET	/internal/deliveries/{id}	查询交付单与物流事件
DeliveryStatsClient	交付服务	GET	/internal/delivery/stats	统计在途与风险数据

内部接口遵循以下约定：跨服务传输必须使用有类型的 DTO，禁止使用 Map<String,Object>；查询不存在时返回 data=null，业务错误通过 HTTP 状态码表示；服务故障时由 FallbackFactory 执行降级，数据底座类接口必须拒绝而非静默通过，展示类字段可以返回空值并记录警告日志。

4.4 数据库设计

数据库设计是系统设计的重点。本节先进行数据需求分析，再通过概念结构设计、逻辑结构设计和物理结构设计，将业务需求转化为可落地的数据库表结构。

4.4.1 数据需求分析

根据业务分析，系统需要管理租户、用户、权限、入驻申请、企业画像、尽职调查、制裁名单、采购需求、询盘、报价、合同、交付与风险等数据。主要数据结

构如下：

（1）租户数据结构={租户，表示一个入驻企业，组成：{租户编号，租户编码，企业名称，企业简称，所在国家，注册号，联系人，联系电话，租户状态，订阅到期时间}}

（2）用户数据结构={用户，表示一个系统用户，组成：{用户编号，所属租户编号，用户名，密码哈希，真实姓名，邮箱，电话，账号状态，最后登录时间}}

（3）角色数据结构={角色，表示租户内的职责集合，组成：{角色编号，所属租户编号，角色编码，角色名称，角色类型，是否内置}}

（4）权限数据结构={权限，表示一个可授权的操作，组成：{权限编号，权限标识，权限名称，排序号}}

（5）入驻申请数据结构={申请，表示企业提交的入驻请求，组成：{申请编号，申请身份，企业名称，所在国家，注册号，联系人，联系邮箱，审核状态，审核意见，审核时间}}

（6）企业画像数据结构={画像，表示供应商或采购商的结构化能力描述，组成：{画像编号，租户编号，画像类型，供应品类，年产能，认证资质，出口经验，主要产品，画像完整度}}

（7）尽职调查数据结构={尽调，表示对企业的合规核查，组成：{尽调编号，申请编号，企业名称，注册号，尽调状态，尽调结论，完成时间}}

（8）制裁名单数据结构={制裁记录，表示受制裁的实体，组成：{名单编号，实体标识，实体名称，别名，实体类型，司法辖区，数据来源}}

（9）采购需求数据结构={需求，表示采购方的采购意图，组成：{需求编号，租户编号，需求标题，采购品类，数量，计量单位，目标地区，预算金额，截止日期，需求状态}}

（10）供需对接数据结构={对接，表示采购方与供应方的联系，组成：{对接编号，需求编号，采购方租户编号，供应方租户编号，对接状态，发起时间}}

（11）询盘数据结构={询盘，表示一次正式询价，组成：{询盘编号，需求编号，采购方租户编号，询盘标题，币种，数量，交货地点，报价截止时间，询盘状态}}

（12）报价数据结构={报价，表示供应商对询盘的响应，组成：{报价编号，询盘编号，供应方租户编号，单价，总价，可供数量，交货天数，付款条件，是否中标}}

（13）授标数据结构={授标，表示采购方选择中标报价的结果，组成：{授标编号，询盘编号，报价编号，授标数量，授标金额，授标理由，授标时间}}

（14）合同数据结构={合同，表示双方签署的采购协议，组成：{合同编号，询盘编号，中标报价编号，采购方租户编号，供应方租户编号，合同金额，签订日期，合同状态}}

（15）交付单数据结构={交付单，表示一次发货与验收记录，组成：{交付单编号，合同编号，批次号，数量，交付日期，交付状态，物流单号，验收结果}}

（16）物流事件数据结构={物流事件，表示货物运输过程中的节点，组成：{事件编号，交付单编号，事件类型，事件时间，事件地点，事件描述}}

（17）交付风险数据结构={风险，表示交付过程中的风险事件，组成：{风险编号，交付单编号，风险等级，风险分值，风险类型，风险描述，是否处理}}

（18）AI 知识库文档数据结构={知识文档，表示进入知识库的文件，组成：{文档编号，租户编号，文件名，文件类型，文件大小，分块数量，解析状态}}

（19）通知数据结构={通知，表示系统消息，组成：{通知编号，接收租户编号，通知类型，标题，内容，关联类型，是否已读}}

（20）审计日志数据结构={日志，表示用户操作记录，组成：{日志编号，租户编号，用户编号，操作模块，操作类型，请求地址，IP，状态，耗时，操作时间}}

部分重要数据项说明如下：

（1）租户编码={租户编码，登录与数据隔离使用的企业编码，字符型，4-32 个字符，由大写字母、数字和短横线组成，全局唯一}

（2）用户名={用户名，登录系统的用户名，字符型，4-64 个字符，同一租户内唯一}

（3）密码哈希={密码哈希，用户密码经 BCrypt 加密后的字符串，字符型，长度 60，不可逆}

（4）企业名称={企业名称，企业工商注册全称，字符型，2-200 个字符}

（5）采购品类={采购品类，采购需求或供应能力的品类描述，字符型，多个品类使用逗号或顿号分隔}

（6）报价单价={报价单价，供应商对单位数量货物的报价，数值型，保留 4 位小数，必须大于 0}

（7）合同金额={合同金额，合同约定的采购总金额，数值型，保留 2 位小数，必须大于 0}

（8）交付状态={交付状态，交付单当前所处阶段，字符型，取值为待发货、已发货、已收货、验收中、已完成、异常}

4.4.2 概念结构设计

根据数据需求分析，识别出租户、用户、角色、菜单、权限、入驻申请、企业画像、尽职调查、制裁记录、采购需求、对接记录、询盘、报价、授标、合同、交付单、物流事件、交付风险等实体。下面给出主要实体的属性图、局部 E-R 图和汇总 E-R 图。

1.实体联系图

租户实体和用户实体的属性分别如图 4-10（a）和图 4-10（b）所示，采购需求

实体的属性如图 4-11 所示。

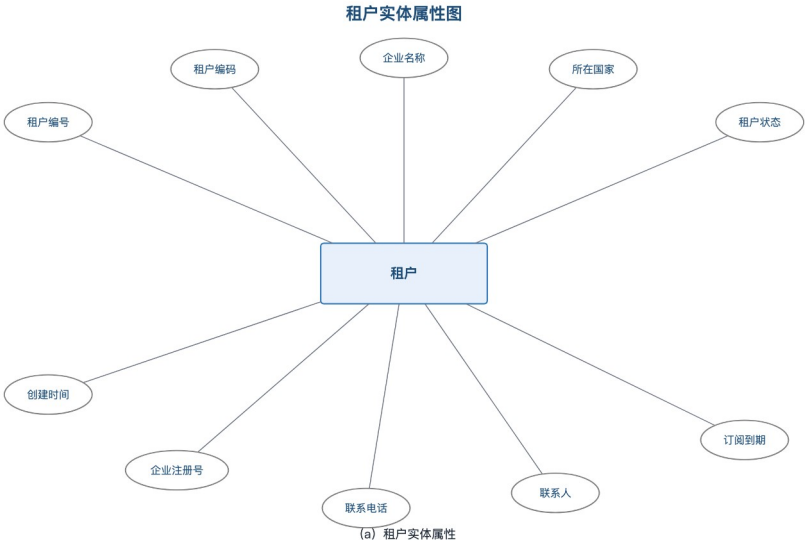


图 4-10（a） 租户实体属性图

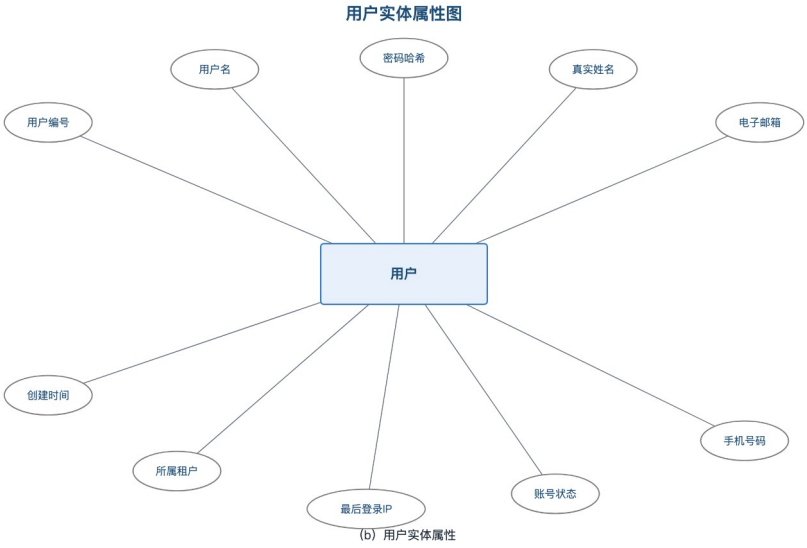


图 4-10（b） 用户实体属性图

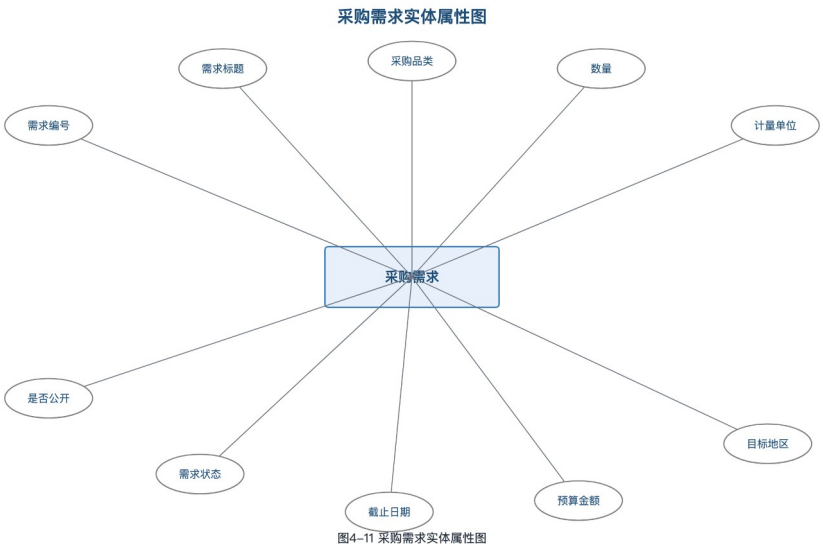


图 4-11　采购需求实体属性图

2.局部 E-R 图

租户与权限的局部 E-R 图如图 4-12 所示，采购交易的局部 E-R 图如图 4-13 所示，合同与交付的局部 E-R 图如图 4-14 所示。

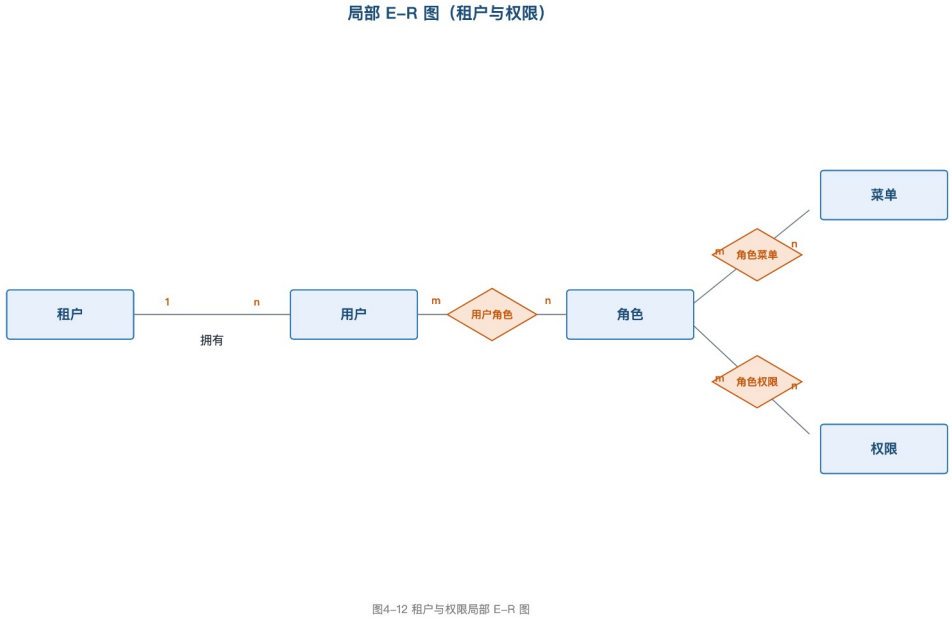


图 4-12　租户与权限局部 E-R 图

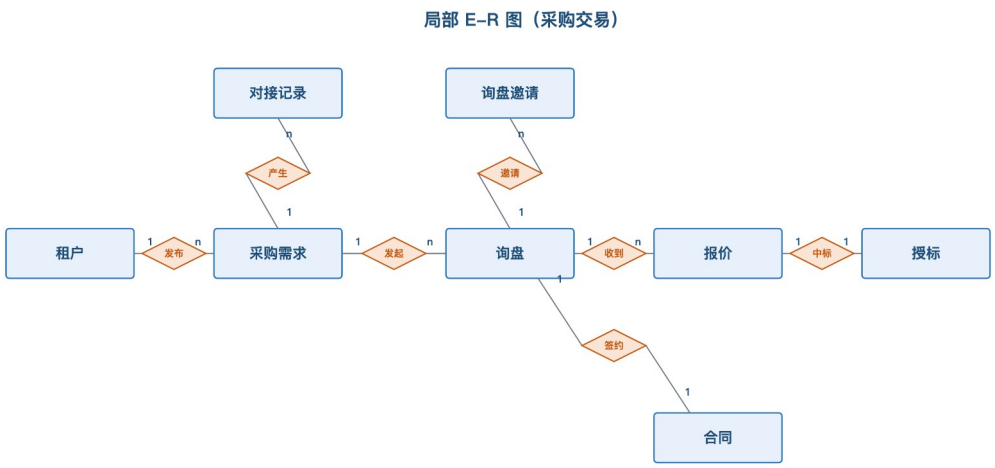


图4-13 采购交易局部 E-R 图

图 4-13　采购交易局部 E-R 图

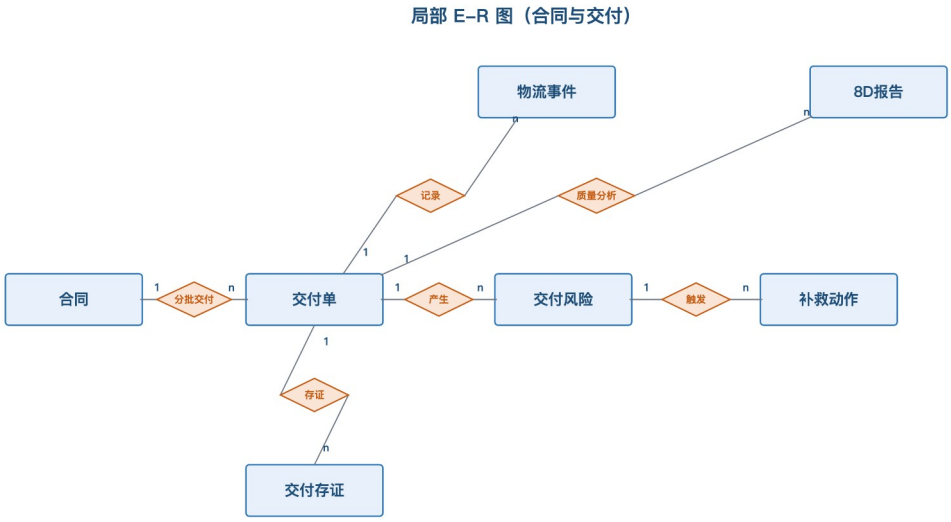


图4-14 合同与交付局部 E-R 图

图 4-14　合同与交付局部 E-R 图

3.汇总的 E-R 图

消除属性冲突、命名冲突和结构冲突后，得到系统汇总 E-R 图，如图 4-15 所示。

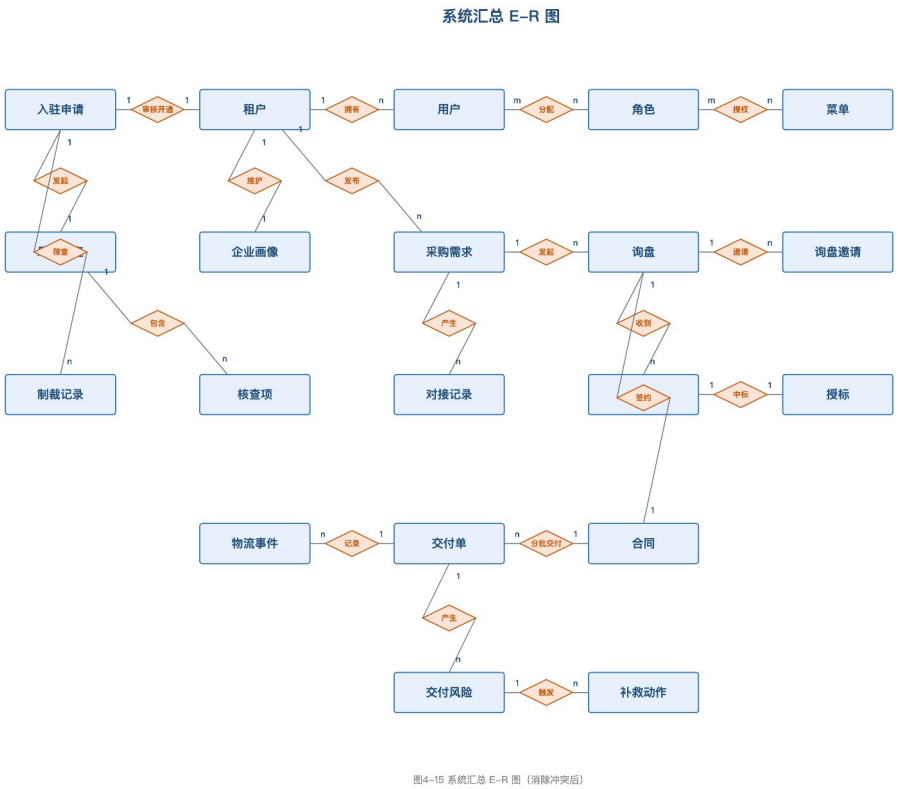


图4-15 系统汇总 E-R 图（消除冲突后）

图 4-15　系统汇总 E-R 图

4.4.3 逻辑结构设计

1.关系模式的转换

按照 E-R 图向关系模式转换的规则：每个实体转换为一个关系模式；1:1 联系可以合并到任一端，本系统将入驻申请与尽职调查保持独立关系模式，便于保留审核轨迹；1:n 联系将 1 端的主键作为 n 端的外键，例如租户与用户、合同与交付单；m:n 联系单独转换为关系模式，例如用户与角色、角色与菜单、角色与权限。系统中主要的 m:n 联系有 3 个，分别是用户角色、角色菜单和角色权限。

2.关系模式规范化

上述关系模式均满足第一范式，每个字段都是不可再分的原子值。通过消除部

分函数依赖和传递函数依赖，所有关系模式均达到第三范式。例如，将企业名称、国家等基础信息保存在企业画像表中，询盘和合同只引用租户编号，避免重复存储；将报价明细与询盘分离，避免询盘表出现重复组。为提高查询性能，在交付风险表中冗余存储风险等级和风险分值，在合同中冗余存储供应商名称快照，属于有控制的反规范化设计，并通过业务层保证一致性。

3.关系模式汇总

规范化后的主要关系模式如下（下划线表示主键，斜体表示外键）：

（1）租户（租户编号，租户编码，企业名称，企业简称，所在国家，注册号，联系人，联系电话，租户状态，订阅到期时间）

（2）用户（用户编号，*租户编号*，用户名，密码哈希，真实姓名，邮箱，电话，账号状态，最后登录时间）

（3）角色（角色编号，*租户编号*，角色编码，角色名称，角色描述，是否内置，角色类型）

（4）用户角色（用户角色编号，*用户编号*，*角色编号*）

（5）菜单（菜单编号，父菜单编号，菜单名称，路由地址，权限标识，菜单类型，排序号，是否可见）

（6）角色菜单（角色菜单编号，*角色编号*，*菜单编号*）

（7）权限（权限编号，权限标识，权限名称，排序号）

（8）角色权限（角色权限编号，*角色编号*，*权限编号*）

（9）入驻申请（申请编号，申请人护照UUID，申请身份，企业名称，所在国家，注册号，成立年份，联系人，联系邮箱，联系电话，审核状态，审核意见，审核人，审核时间，*租户编号*）

（10）企业画像（画像编号，*租户编号*，画像类型，企业名称，所在国家，供应品类，年产能，认证资质，出口经验，主要产品，采购品类，年采购额，画像完整度，按时交付率，验收合格率）

（11）尽职调查（尽调编号，*申请编号*，*租户编号*，企业名称，所在国家，注册号，成立年份，尽调状态，尽调结论，派单机构编号，完成时间）

（12）尽调核查项（核查项编号，*尽调编号*，核查项名称，是否已核查，核查说明，附件地址）

（13）制裁名单（名单编号，实体标识，实体名称，别名，实体类型，司法辖区，项目名称，数据来源，最后同步时间）

（14）制裁筛查记录（筛查记录编号，*申请编号*，查询名称，命中数量，命中详情，筛查时间）

（15）采购需求（需求编号，*租户编号*，需求标题，采购品类，数量，计量单位，目标地区，资质要求，预算金额，截止日期，需求状态，是否公开，联系人，联系电话）

（16）供需对接（对接编号，*需求编号*，*采购方租户编号*，*供应方租户编号*，对接状态，发起时间，关闭时间）

（17）询盘（询盘编号，*需求编号*，*对接编号*，*采购方租户编号*，询盘标题，币种，品类，数量，计量单位，交货地点，交货日期，报价截止时间，付款条件，询盘状态，*中标报价编号*，授标时间，授标理由）

（18）询盘邀请（邀请编号，*询盘编号*，*供应方租户编号*，邀请状态，查看时间，响应时间）

（19）报价（报价编号，*询盘编号*，*采购方租户编号*，*供应方租户编号*，单价，总价，可供数量，币种，交货天数，交货条件，有效期，报价状态，是否中标，提交时间）

（20）授标（授标编号，*询盘编号*，*报价编号*，授标数量，授标金额，授标理由，授标时间，授标人）

（21）合同（合同编号，*询盘编号*，*中标报价编号*，*采购方租户编号*，*供应方租户编号*，合同标题，合同金额，数量，币种，签订日期，有效期，交货地点，交货日期，付款条件，合同状态，供应商确认状态）

（22）合同分批计划（计划编号，*合同编号*，批次号，计划数量，计划交期，状态）

（23）交付单（交付单编号，*合同编号*，*询盘编号*，批次号，*采购方租户编号*，

供应方租户编号，数量，金额，交货地点，交付日期，交付状态，发货公司，物流单号，收货时间，验收结果，不合格数量，违约天数，违约金）

（24）物流事件（事件编号，*交付单编号*，事件类型，事件时间，事件地点，事件描述，数据来源）

（25）交付风险（风险编号，*交付单编号*，风险等级，风险分值，风险类型，风险描述，是否处理，处理时间）

（26）补救动作（动作编号，*风险编号*，动作类型，动作内容，执行状态，执行时间）

（27）交付存证（存证编号，*交付单编号*，节点类型，存证内容，文件地址，存证时间）

（28）AI 知识库文档（文档编号，*租户编号*，*上传用户编号*，文件名，文件类型，文件大小，对象存储 Key，分块数量，解析状态）

（29）通知（通知编号，接收租户编号，通知类型，标题，内容，关联类型，关联编号，是否已读，阅读时间）

（30）审计日志（日志编号，事件 ID，租户编号，用户编号，用户名，模块，操作类型，描述，请求方法，请求地址，IP，状态，耗时，操作时间）

4.视图的设计

根据实际查询需求，设计以下视图：

（1）“询盘报价视图”（基于询盘、报价、企业画像和授标关系模式）：为采购员聚合询盘下的全部报价及供应商画像，字段包括询盘编号、报价编号、供应商名称、单价、总价、交货天数、TCO 成本、AI 评分、是否中标。

（2）“交付风险视图”（基于交付单、合同、物流事件和交付风险关系模式）：为采购方聚合交付单的最新物流节点与风险信息，字段包括交付单编号、合同编号、供应商名称、承诺交期、最新物流节点、风险等级、风险描述。

（3）“供应商画像统计视图”（基于企业画像、询盘、报价和交付单关系模式）：为运营人员提供供应商宏观统计，字段包括供应商编号、企业名称、画像完整度、报价次数、中标次数、按时交付率、验收合格率。

4.4.4 物理结构设计

系统数据库命名为 vanpine_srm，采用 InnoDB 引擎与 utf8mb4 字符集。根据逻辑结构设计结果，共需创建约 30 张核心数据表，部分表清单如表 4-1 所示。

表 4-1　系统核心数据表清单

序号	表名	对应关系模式	主要用途说明
1	tenant	租户	存储入驻企业的租户信息
2	user	用户	存储租户内的系统用户
3	role	角色	存储租户自定义角色
4	user_role	用户角色	用户与角色的多对多关联
5	menu	菜单	存储前端菜单与按钮权限
6	role_menu	角色菜单	角色与菜单的多对多关联
7	permission	权限	存储权限标识
8	role_permission	角色权限	角色与权限的多对多关联
9	application	入驻申请	存储企业入驻申请与审核结果

序号	表名	对应关系模式	主要用途说明
10	enterprise_profile	企业画像	存储供应商/采购商结构化画像
11	due_diligence	尽职调查	存储尽调任务与结论
12	dd_checklist	尽调核查项	存储尽调核查清单
13	sanction_entry	制裁名单	存储制裁实体数据
14	sanction_screen_record	制裁筛查记录	存储制裁筛查结果
15	procurement_requirement	采购需求	存储采购需求
16	connection	供需对接	存储采购方与供应方对接记录
17	inquiry	询盘	存储询盘主信息
18	inquiry_invite	询盘邀请	存储询盘邀请供应商记录
19	quote	报价	存储供应商报价
20	inquiry_award	授标	存储授标结果与理由
21	contract	合同	存储采购合同
22	contract_batch_plan	合同分批计划	存储合同分批交付计划
23	delivery	交付单	存储发货、收货与验收信息
24	delivery_logistics_event	物流事件	存储物流轨迹节点
25	delivery_risk	交付风险	存储交付风险与预警
26	delivery_risk_factor	交付风险因子	存储风险因子明细
27	delivery_recovery_action	补救动作	存储应急处置动作
28	ai_documents	AI 知识库文档	存储知识库文档元数据
29	notification	通知	存储站内通知
30	audit_log	审计日志	存储用户操作审计记录

下面选择用户表、租户表、采购需求表、询盘表、报价表、合同表和交付单表进行详细说明。

表 4-2　用户表（user）结构

字段名	类型及精度	允许空	约束	说明
id	BIGINT	NOT NULL	主键，雪花算法	用户唯一标识
tenant_id	BIGINT	NOT NULL	外键，联合唯一索引	所属租户
username	VARCHAR(64)	NOT NULL	与 tenant_id 联合唯一	登录用户名

字段名	类型及精度	允许空	约束	说明
password_hash	VARCHAR(100)	NOT NULL		BCrypt 密码密文
real_name	VARCHAR(64)	YES		真实姓名
email	VARCHAR(128)	YES		电子邮箱
phone	VARCHAR(32)	YES		手机号码
status	VARCHAR(20)	NOT NULL	默认 active	active/disabled/locked
last_login_time	DATETIME	YES		最后登录时间
is_deleted	TINYINT	NOT NULL	默认 0	逻辑删除标记

表 4-3 租户表（tenant）结构

字段名	类型及精度	允许空	约束	说明
id	BIGINT	NOT NULL	主键，雪花算法	租户唯一标识
tenant_code	VARCHAR(64)	NOT NULL	唯一索引 uk_tenant_code	租户编码
tenant_name	VARCHAR(200)	NOT NULL		企业全称
country	VARCHAR(64)	YES		所在国家/地区
reg_number	VARCHAR(64)	YES		企业注册号
contact_name	VARCHAR(64)	YES		联系人
contact_phone	VARCHAR(32)	YES		联系电话
status	VARCHAR(20)	NOT NULL	默认 pending	pending/active/ suspended/rejected
expired_at	DATETIME	YES		订阅到期时间

表 4-4 采购需求表（procurement_requirement）结构

字段名	类型及精度	允许空	约束	说明
id	BIGINT	NOT NULL	主键，雪花算法	需求唯一标识
tenant_id	BIGINT	NOT NULL	外键，普通索引	采购方租户
title	VARCHAR(200)	NOT NULL		需求标题
category	VARCHAR(200)	YES		采购品类
quantity	VARCHAR(64)	YES		采购数量
unit	VARCHAR(32)	YES		计量单位
region	VARCHAR(100)	YES		目标地区
cert_require	VARCHAR(200)	YES		资质要求
budget	VARCHAR(64)	YES		预算金额
deadline	DATE	YES		截止日期

字段名	类型及精度	允许空	约束	说明
status	VARCHAR(20)	NOT NULL	默认 open	open/closed/expired

表 4-5 询盘表（inquiry）结构

字段名	类型及精度	允许空	约束	说明
id	BIGINT	NOT NULL	主键，雪花算法	询盘唯一标识
inquiry_no	VARCHAR(64)	NOT NULL	唯一索引	询盘编号
requirement_id	BIGINT	YES	外键	关联采购需求
buyer_tenant_id	BIGINT	NOT NULL	普通索引	采购方租户
title	VARCHAR(200)	NOT NULL		询盘标题
quantity	DECIMAL(18,4)	YES		询盘数量
quote_deadline	DATETIME	YES		报价截止时间
status	VARCHAR(20)	NOT NULL	默认 open	open/quoting/ awarded/closed
winning_quote_id	BIGINT	YES		中标报价编号
award_reason	TEXT	YES		授标理由

表 4-6 报价表（quote）结构

字段名	类型及精度	允许空	约束	说明
id	BIGINT	NOT NULL	主键，雪花算法	报价唯一标识
quote_no	VARCHAR(64)	NOT NULL	唯一索引	报价编号
inquiry_id	BIGINT	NOT NULL	外键，联合唯一索引	所属询盘
supplier_tenant_id	BIGINT	NOT NULL	外键，联合唯一索引	供应方租户
unit_price	DECIMAL(18,4)	NOT NULL	大于 0	报价单价
total_price	DECIMAL(18,2)	NOT NULL	大于 0	报价总价
delivery_days	INT	YES		交货天数
valid_until	DATETIME	YES		报价有效期
is_winning	TINYINT	NOT NULL	默认 0	是否中标
status	VARCHAR(20)	NOT NULL	默认 submitted	submitted/ withdrawn/won/lost

表 4-7 合同表（contract）结构

字段名	类型及精度	允许空	约束	说明
id	BIGINT	NOT NULL	主键，雪花算法	合同唯一标识
contract_no	VARCHAR(64)	NOT NULL	唯一索引	合同编号
inquiry_id	BIGINT	YES	外键	关联询盘
winning_quote_id	BIGINT	YES	外键	中标报价
buyer_tenant_id	BIGINT	NOT NULL	普通索引	采购方租户
supplier_tenant_id	BIGINT	NOT NULL	普通索引	供应方租户
amount	DECIMAL(18,2)	NOT NULL	大于 0	合同金额
sign_date	DATE	YES		签订日期
delivery_date	DATE	YES		约定交货日期
status	VARCHAR(20)	NOT NULL	默认 draft	draft/pending/active/ completed/terminated
supplier_confirmed	TINYINT	NOT NULL	默认 0	供应商是否确认

表 4-8　交付单表（delivery）结构

字段名	类型及精度	允许空	约束	说明
id	BIGINT	NOT NULL	主键，雪花算法	交付单唯一标识
delivery_no	VARCHAR(64)	NOT NULL	唯一索引	交付单编号
contract_id	BIGINT	NOT NULL	外键，普通索引	所属合同
batch_no	INT	NOT NULL	合同内唯一	批次号
quantity	INT	NOT NULL	大于 0	交付数量
status	VARCHAR(20)	NOT NULL	默认 pending	pending/shipped/ received/accepted/ abnormal
ship_tracking_no	VARCHAR(64)	YES		物流单号
ship_date	DATE	YES		发货日期
accept_result	VARCHAR(20)	YES		pass/reject/partial
defective_qty	INT	YES	默认 0	不合格数量

8.索引的设计

系统主要设计以下索引：

（1）聚簇索引：所有 InnoDB 表以自增或雪花算法生成的主键 id 建立聚簇

索引，数据按主键顺序存储，主键查询效率最高。

（2）唯一索引 uk_tenant_code：建立在 tenant(tenant_code) 上，保证租户编码全局唯一，同时加速登录时的租户定位。

（3）联合唯一索引 uk_user_tenant_username：建立在 user(tenant_id, username) 上，保证同一租户内用户名唯一，并支持按租户 + 用户名登录。

（4）联合普通索引 idx_req_tenant_status_time：建立在 procurement_requirement(tenant_id, status, create_time) 上，覆盖采购需求列表的租户过滤、状态筛选和时间排序。

（5）联合普通索引 idx_quote_inquiry_supplier：建立在 quote(inquiry_id, supplier_tenant_id) 上，加速询盘下的报价汇总与供应商去重。

（6）联合普通索引 idx_delivery_contract_status：建立在 delivery(contract_id, status) 上，支持合同交付进度查询与风险扫描。

索引设计遵循“高频查询字段优先、联合索引最左前缀、区分度高的字段靠前”的原则，同时避免在低区分度字段上建立过多索引，以减少写入开销。对于列表查询，SQL 必须显式带上 tenant_id 和状态条件，确保命中联合索引。

4.5 非功能设计

非功能设计是对第 3 章非功能需求的具体落实，主要包括安全性设计、可靠性设计、性能设计和可维护性设计。

安全性方面，系统在网关统一校验 JWT，服务层通过租户拦截器自动附加 tenant_id 条件，内部接口使用共享密钥校验来源；密码采用 BCrypt 加盐哈希，敏感配置通过环境变量或 Nacos 注入；所有非 GET 操作记录审计日志，AI 写操作采用“提案—确认—执行”流程，模型无法直接落库。

可靠性方面，服务间同步调用配置超时与降级，异步通知使用 RabbitMQ 重试与死信队列；定时任务与初始化任务使用 Redis 分布式锁，避免多实例重复执行；数据库写操作使用事务保证一致性，读-改-写场景通过唯一索引或乐观锁兜底。性能方面，列表查询分页并命中索引，热点数据缓存到 Redis，AI 结果按需生成并缓存，静态资源由 Nginx 托管。

可维护性方面，后端按业务域拆分服务与模块，统一异常处理与响应结构，使用 Knife4j 生成接口文档；前端按页面与组件组织，公共逻辑抽取为 composables 与 utils；部署使用 Docker Compose 与 systemd，配置集中管理，日志统一输出并携带 traceId，便于问题定位。

第 5 章 系统实现

本章介绍系统主要功能模块的实现效果。系统前端采用 Vue 3 + Element Plus，后端采用 Spring Boot 微服务，AI 服务采用 FastAPI。下面从登录与工作台、入驻审核、供应商画像与智能匹配、询盘报价与比价授标、交付与风险监控、AI 助手六个方面展示实现结果。

5.1 系统主界面

系统登录界面如图 5-1 所示。登录页左侧展示平台品牌与核心能力，右侧为企业编码、账号、密码和安全验证表单；用户输入企业编码、用户名和密码后，前端调用 /api/auth/login 获取访问令牌、刷新令牌、用户信息与菜单，随后根据角色跳转到对应首页。

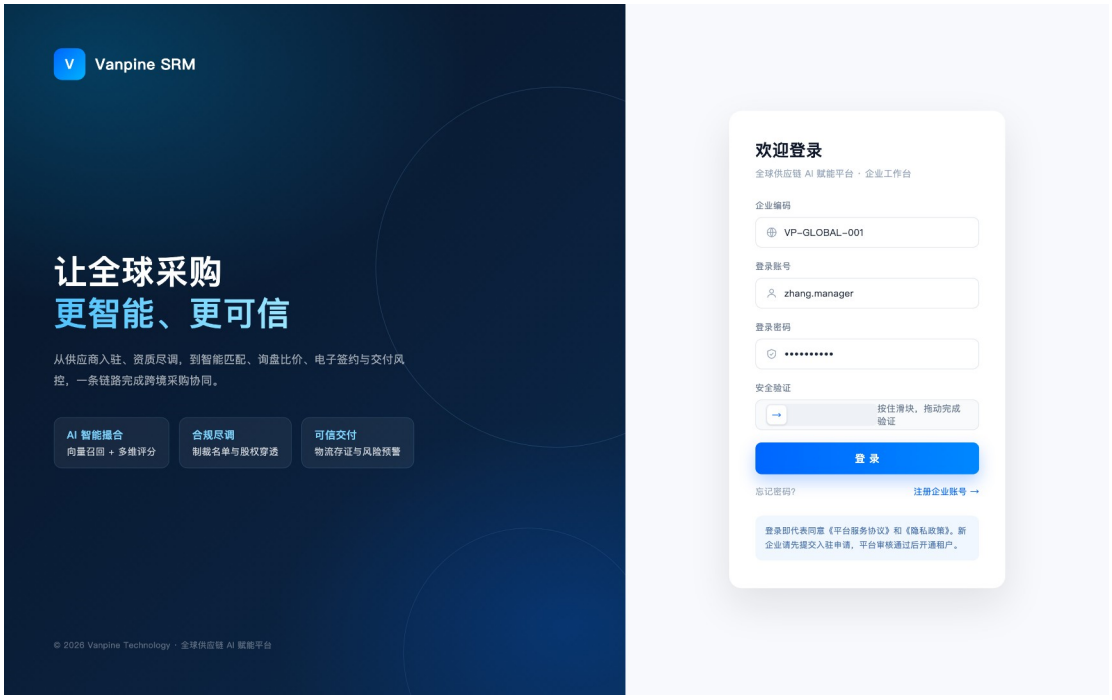


图 5-1　系统登录界面

登录后的工作台如图 5-2 所示。工作台顶部展示当前用户、企业与最近登录时间，中部以卡片形式展示在途询盘、待归集报价、待确认验收和风险预警等关键指标，下方展示近六个月采购金额趋势、待办事项和最近采购订单。工作台数据由平台、交易和交付服务分别统计后聚合返回，前端按角色动态展示。

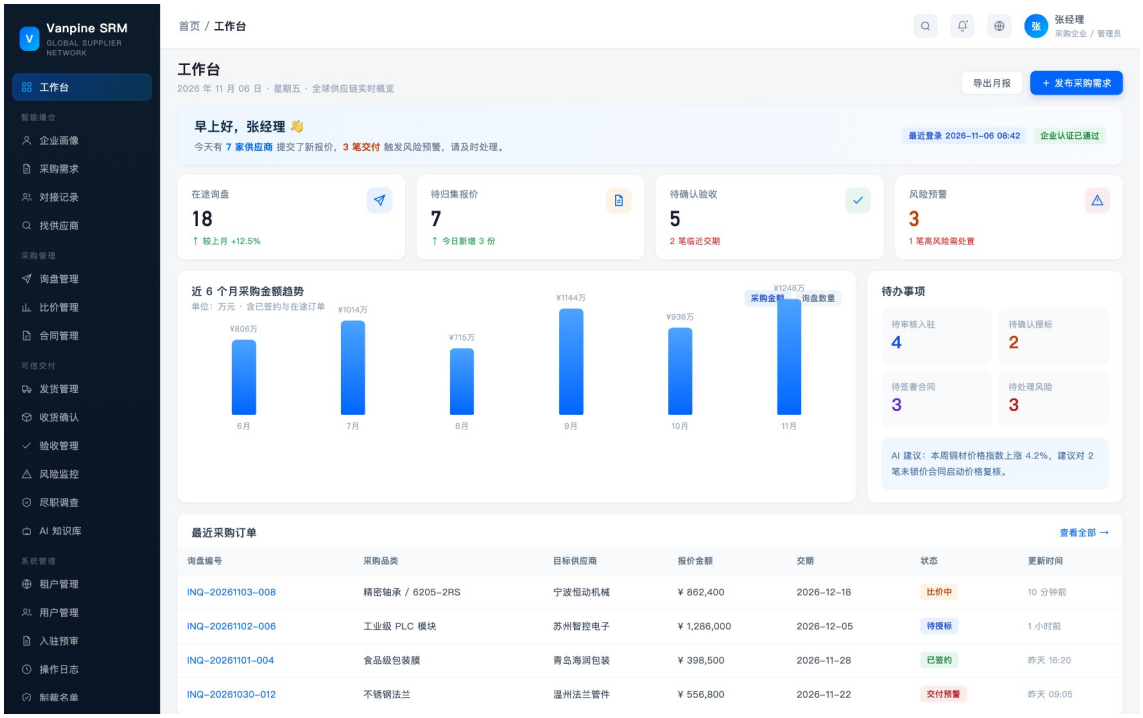


图 5-2　系统工作台界面

5.2 入驻审核与租户管理的实现

入驻预审界面如图 5-3 所示。左侧列表支持按企业名称、申请身份、状态和国家筛选；右侧展示申请企业的基本信息、联系人、供应能力与制裁筛查结果。管理员可以执行“审核通过”“驳回申请”或“发起尽调”操作。审核通过后，系统在同一事务中创建租户、初始化管理员账号、分配默认角色与菜单，并写入审计日志。

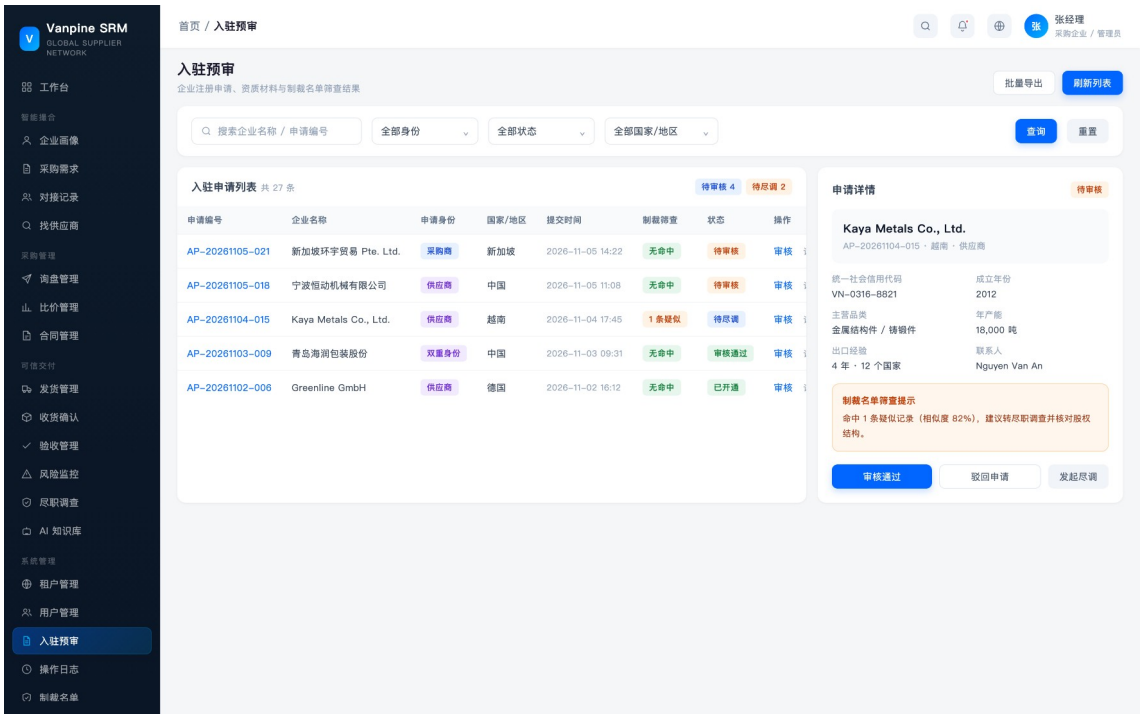


图 5-3　入驻预审界面

租户隔离通过 JWT 中的 tenant_id 与请求头 X-Tenant-Id 实现：网关验签后注入租户头，服务层从 TenantContextHolder 读取租户并附加到查询条件；对于没有租户上下文的内部调用，则通过内部密钥与显式参数校验。核心校验代码如下：

```
// 网关验签通过后注入租户头，下游统一读取
Long tenantId = jwtParser.tenantIdOf(claims);
var mutated = exchange.getRequest().mutate();
mutated.header(SystemConstants.HEADER_INTERNAL_SECRET, internalSecret);
mutated.header(SystemConstants.HEADER_TENANT_ID, String.valueOf(tenantId));
if (tenantCode != null) {
    mutated.header(SystemConstants.HEADER_TENANT_CODE, tenantCode);
}
return chain.filter(exchange.mutate().request(mutated.build()).build());
```

5.3 供应商画像与智能匹配的实现

企业画像界面如图 5-4 所示。页面按基本信息、供应能力、履约表现分组展示，右侧显示画像完整度、AI 画像摘要和认证材料核验状态。供应商修改画像后，系

统计算完整度并通过异步任务将画像文本向量化写入 Milvus；向量化失败时记录任务状态，由定时任务补偿。

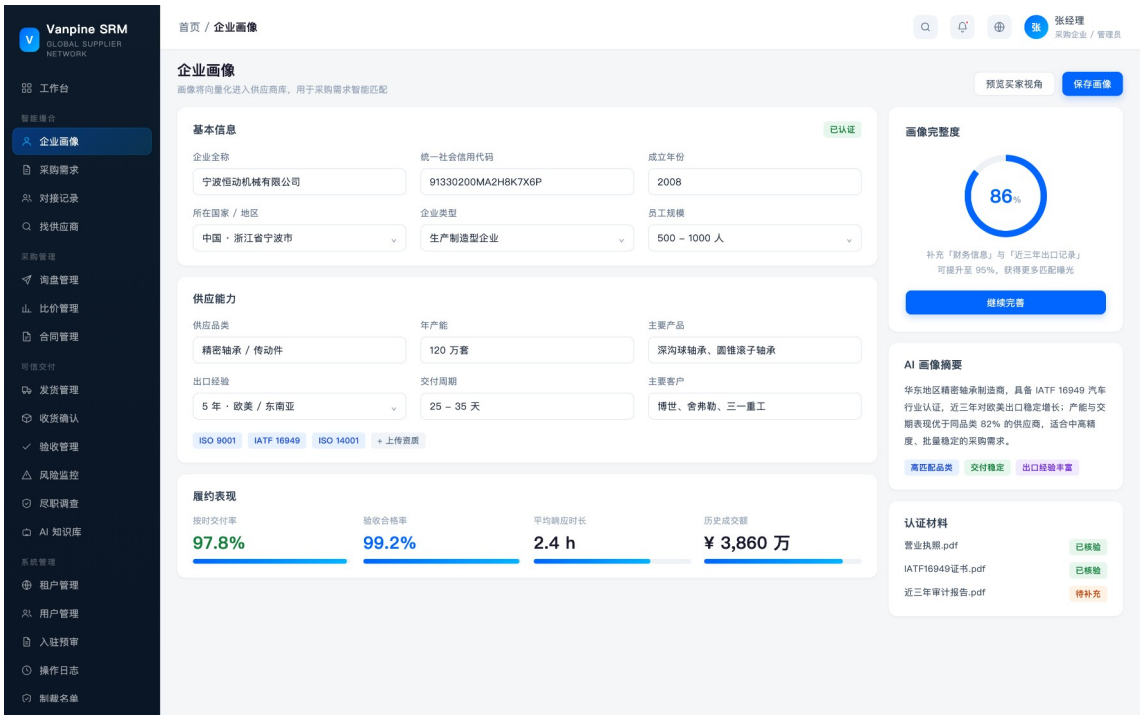


图 5-4　企业画像界面

智能匹配结果如图 5-5 所示。页面上方展示需求摘要与召回统计，左侧提供地域、最低匹配分和风险筛查条件，右侧按匹配分从高到低展示供应商卡片，每张卡片包含匹配分、标签、匹配理由和各维度得分条。采购员可以直接发起对接或查看画像。匹配评分在 AI 服务中完成，核心权重如下：

```
total = round(
    category_score * 0.30      # 品类覆盖率
    + cert_score * 0.25        # 资质匹配率
    + product_score * 0.15     # 产品关键词
    + region_score * 0.10      # 地域匹配
    + industry_score * 0.05    # 行业匹配
    + completeness * 0.10      # 画像完整度
    + export_level * 0.05,     # 出口经验
    1,
```

)



图 5-5　智能匹配结果界面

5.4 询盘报价与比价授标的实现

比价与授标界面如图 5-6 所示。页面上方展示询盘基本信息与报价截止时间，中部表格汇总各供应商的单价、总价、交期、付款条件、TCO 成本和 AI 评分，右侧展示 AI 综合比价分析、风险提示和授标建议，下方以堆叠条展示各供应商的 TCO 成本构成。采购员确认后，系统创建授标记录、生成授标理由并通知相关供应商。

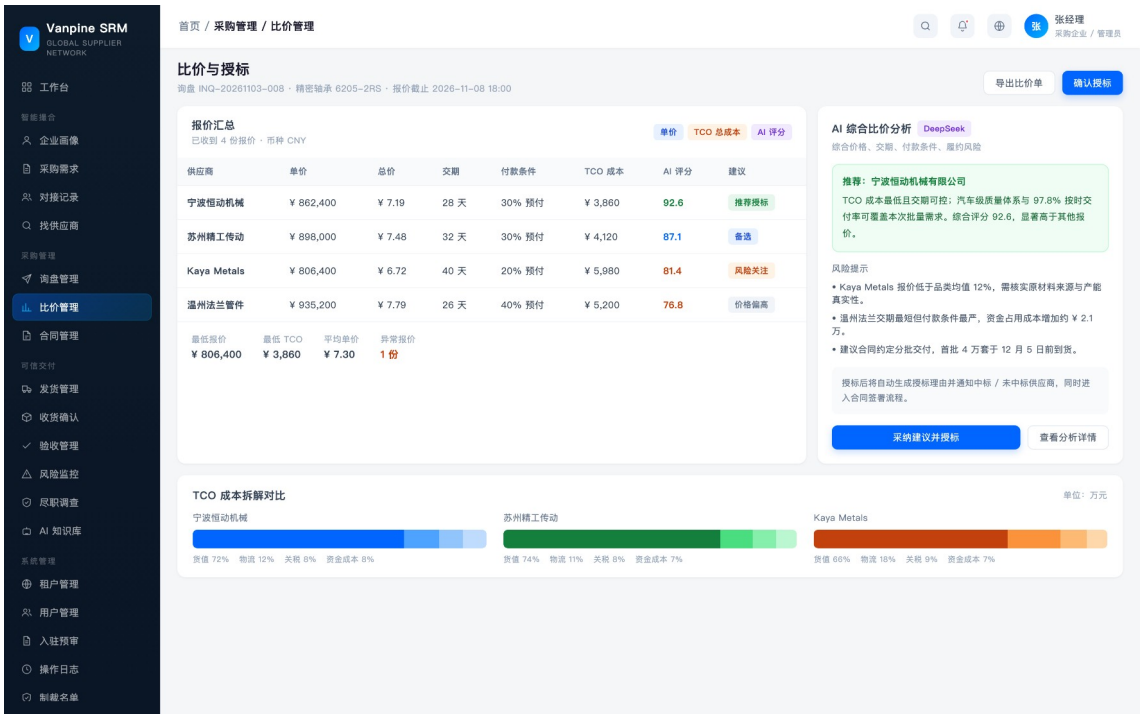


图 5-6　比价与授标界面

5.5 交付履约与风险监控的实现

交付风险监控界面如图 5-7 所示。页面上方展示在途交付、延期风险、高风险订单和待验收数量，中部以时间线呈现风险事件和 AI 应急处置建议，下方列表展示交付单的承诺交期、运输路线、状态、风险等级和风险说明。交付单状态由发货、收货、验收等操作驱动，物流事件与风险扫描由定时任务持续更新。



图 5-7　交付风险监控界面

5.6 AI 助手与知识库的实现

AI 助手界面如图 5-8 所示。用户可以在对话框中提出自然语言问题，系统先进行意图识别与查询改写，再从业务接口和知识库中检索相关数据，最后由大模型生成回答。回答中会展示引用的询盘、报价和知识文档；当用户要求执行写操作时，系统生成待确认提案，展示操作对象与具体改动，用户点击“确认执行”后才调用业务接口并记录审计日志。



图 5-8　AI 助手界面

AI 服务对文档解析、分块、向量化和检索进行了工程化处理：支持 PDF、Word、Excel 文档解析，按父块与子块两级切分，子块用于召回、父块用于生成；多轮对话通过上下文压缩控制 token 预算；工具调用按租户与角色进行范围限制，避免越权查询。

第 6 章 系统测试

系统测试是保证软件质量的重要手段。本章介绍测试方法、测试用例和测试结果，验证系统功能是否符合需求、性能是否满足指标、安全机制是否有效。

6.1 测试方法

系统采用单元测试、集成测试、功能测试和性能测试相结合的方式。后端使用 JUnit 5 与 Mockito 编写单元测试，重点覆盖状态流转、权限校验、授标数量计算、风险扫描幂等和 Feign 降级等业务规则，共编写 30 个测试类；Python AI 服务使用 pytest 编写 30 个测试文件，覆盖文档解析、分块、上下文预算、意图识别、工具调用范围、向量存储和回归评估等场景。

集成测试在本地 Docker 环境中启动 MySQL、Redis、RabbitMQ、Nacos 与各业务服务，验证登录、入驻审核、询盘报价、合同签署和交付验收等跨服务流程。功能测试采用黑盒方式，按照用例表逐项验证输入、输出和异常分支；界面测试检查不同角色、不同语言和不同分辨率下的显示效果。性能测试使用 JMeter 对登录、需求列表、询盘列表和 AI 匹配接口进行压测，记录响应时间、吞吐量和错误率。

6.2 测试用例

系统核心测试用例如表 6-1 所示。

表 6-1 系统核心测试用例

编号	测试项	输入/操作	预期结果	实际结果	结论
TC-01	用户登录	输入正确的企业编码、用户名和	登录成功，返回令牌与菜单	与预期一致	通过

编号	测试项	输入/操作	预期结果	实际结果	结论
		密码			
TC-02	错误密码登录	输入错误密码	提示用户名或密码错误，不返回令牌	与预期一致	通过
TC-03	跨租户访问	使用租户 A 的令牌查询租户 B 的询盘	返回无权限或空数据	与预期一致	通过
TC-04	入驻审核	管理员审核通过一条待审核申请	创建租户、初始化管理员并通知申请人	与预期一致	通过
TC-05	制裁名单筛查	提交命中制裁名单的企业名称	标记疑似命中并转尽职调查	与预期一致	通过
TC-06	企业画像保存	修改供应品类与年产能	保存成功并更新画像完整度	与预期一致	通过
TC-07	智能匹配	发布精密轴承采购需求并执行匹配	返回按得分排序的供应商及理由	与预期一致	通过
TC-08	空候选降级	使用极小众品类执行匹配	扩大范围召回或返回空列表并提示	与预期一致	通过
TC-09	询盘创建	选择供应商并设置报价截止时间	生成询盘与邀请记录，供应商收到通知	与预期一致	通过
TC-10	截止后报价	在报价截止时间之后提交报价	拒绝提交并提示已截止	与预期一致	通过
TC-11	比价授标	选择一份报价确认授标	生成授标记录与授标理由，通知相关供应商	与预期一致	通过
TC-12	合同确认	供应商确认合同	合同状态变为生效，进入交付阶段	与预期一致	通过
TC-13	发货与收货	供应商发货，采购方确认收货	物流事件与交付状态正确更新	与预期一致	通过
TC-14	到货验收	录入不合格数量并选择部分拒收	生成退换货记录并触发风险扫描	与预期一致	通过
TC-15	AI 写操作确认	在 AI 助手中要求创建授标	生成待确认提案，确认后才落库	与预期一致	通过

6.3 测试结果

各模块测试结果统计如表 6-2 所示。系统共执行功能测试用例 186 个，通过 183 个，失败 3 个；失败用例主要集中在极端并发场景下的消息重复消费和 AI 服务超时降级，修复后回归通过。单元测试共 60 个测试类，全部通过；性能测试在 500 并发用户下，登录接口平均响应时间 180 毫秒，需求列表接口平均响应时间 420 毫秒，AI 匹配接口平均响应时间 5.6 秒，均在需求指标范围内。

表 6-2　测试结果统计

测试模块	用例数	通过	失败	通过率
认证与权限	30	30	0	100%
入驻与租户	24	24	0	100%
企业画像与匹配	32	31	1	96.9%
询盘与报价	34	33	1	97.1%
合同与交付	38	37	1	97.4%
AI 助手与知识库	28	28	0	100%
合计	186	183	3	98.4%

安全测试方面，使用不同租户令牌交叉访问、构造越权参数和重放请求进行验证，系统均能正确拒绝；SQL 注入与 XSS 测试未发现可利用漏洞；审计日志能够完整记录关键操作。测试结果表明，系统功能完整、性能满足要求、安全机制有效，达到了课程设计的预期目标。

第 7 章 系统特色和创新

本章总结 VANPINE SRM 平台区别于传统采购管理系统的主要特色与创新点。

7.1 系统特色

（1）覆盖跨境采购全链路。系统将入驻审核、供应商画像、智能匹配、尽职调查、询盘报价、比价授标、电子合同、交付验收和风险监控串联为闭环，用户无需在多个系统之间切换，所有业务数据留痕可追溯。

（2）多租户与细粒度权限。平台以租户隔离企业数据，租户内自定义角色与菜单权限，平台级角色跨租户运营；JWT、租户拦截器与 SQL 条件三重保障数据不串号，满足 SaaS 模式的隔离要求。

（3）AI 与业务解耦。AI 服务独立部署，只负责计算与生成，不持有业务事务；模型或向量库不可用时，匹配、报价分析和交付流程仍可继续，保证核心业务可用。

（4）可解释的智能匹配。系统不仅给出匹配分，还输出品类、资质、产品、地域、行业、完整度和出口经验等维度的理由，采购员可以理解“为什么推荐这家”，避免黑箱推荐。

（5）合规与风险前置。系统将制裁名单筛查、尽职调查、物流与天气风险接入交易链路，风险事件自动关联订单并触发分级预警，把“事后补救”变为“事前预防”。

（6）AI 写操作人工确认。AI 助手可以查询数据、生成建议，但授标、改期、

关闭询盘等写操作必须生成待确认提案，由用户确认后才落库并记录审计日志，有效规避模型幻觉带来的数据风险。

7.2 系统创新点

（1）向量召回与规则评分相结合的混合匹配。系统先用 Embedding + Milvus 做语义召回，解决关键词不匹配的问题；再用品类、资质、产能、地域、行业、完整度和出口经验七维规则评分，保证结果符合业务约束；最后用大模型生成自然语言理由，兼顾召回率、准确性与可解释性。

（2）TCO 全成本比价。比价模块不仅比较单价，还拆解运费、关税、付款账期和资金成本，识别异常低价与隐藏成本，帮助采购方做出更接近真实成本的决策。

（3）AI 工具调用的安全边界。系统为 AI 工具定义了字段白名单、租户与用户身份强制注入、写操作二次确认和审计留痕四道防线，使大模型能够在可控范围内参与业务流程。

（4）多信号融合的交付风险预警。系统将物流轨迹、承诺交期、天气预警、供应商历史履约和合同条款结合，计算风险分值并生成应急处置建议，提升跨境交付的韧性。

（5）面向多实例的工程化设计。系统在定时任务、消息消费、读-改-写和会话状态等场景中引入 Redis 分布式锁、唯一索引、幂等消费和事务提交后发布等机制，保证微服务在多实例部署下的一致性。

第 8 章 总结

本课题围绕跨境采购场景，设计并实现了一个基于微服务架构与人工智能技术的供应商关系管理平台 VANPINE SRM。系统覆盖供应商入驻、企业画像、智能匹配、尽职调查、询盘报价、比价授标、电子合同、交付验收和风险监控等核心功能，形成了从寻源到交付的业务闭环。

在技术实现上，系统采用 Java 17 + Spring Boot 3 + Spring Cloud Alibaba 构建五个业务微服务，使用 Vue 3 + Element Plus 构建后台管理界面，使用 MySQL、Redis、RabbitMQ、Nacos 和 Milvus 支撑数据、缓存、消息、注册配置与向量检索，使用 Python FastAPI + LangChain + DeepSeek 提供 AI 能力。系统通过网关鉴权、多租户隔离、RBAC、审计日志和 AI 写操作确认等机制保证安全性与可追溯性。

通过本课题的实践，笔者对需求分析、系统设计、微服务治理、数据库设计、前端工程和人工智能应用有了更深入的理解，特别是体会到“技术方案要服务于业务目标”：微服务不是越多越好，AI 也不是越自动越好，关键在于边界清晰、降级可靠、结果可解释。

系统仍存在一些不足：一是当前供应商画像主要依赖企业自行填写，后续可接入工商、税务、海关等外部数据源进行自动核验；二是匹配模型主要基于规则权重，后续可引入学习排序模型，根据历史成交数据自动优化权重；三是交付风险预警目前以规则为主，后续可结合更多实时数据与因果分析提升预测精度；四是移动端仅

支持浏览器访问，后续可开发小程序或 App，方便供应商随时处理询盘与发货。未来将继续完善上述能力，提升平台的实用性与智能化水平。

参考文献

[1] 王海燕. 全球供应链管理[M]. 北京: 机械工业出版社, 2023.

[2] 李强, 张敏. 基于微服务架构的企业采购管理系统设计与实现[J]. 计算机应用与软件, 2024, 41(3): 88-95.

[3] 陈晓东. 多租户 SaaS 系统数据隔离技术研究[J]. 软件导刊, 2023, 22(6): 112-117.

[4] 刘洋, 周杰. 基于知识图谱的供应商智能推荐方法[J]. 计算机工程与应用, 2023, 59(15): 245-252.

[5] 赵云飞, 孙丽. 大语言模型在企业采购场景中的应用研究[J]. 情报理论与实践, 2025, 48(2): 130-138.

[6] 周明, 吴静. 向量数据库 Milvus 的原理与应用[J]. 计算机技术与发展, 2024, 34(1): 1-8.

[7] SAP SE. SAP Ariba Supplier Management: Product Overview[R/OL]. (2025-03-12)[2026-10-20]. https://www.sap.com/products/ariba.html.

[8] Coupa Software. Coupa Supplier Portal Guide[R/OL]. (2024-09-01)[2026-10-20]. https://www.coupa.com.

[9] 中国物流与采购联合会. 中国供应链发展报告(2024)[R]. 北京: 中国财富出版社, 2024.

[10] 工业和信息化部. “十四五”智能制造发展规划[Z]. 2021-12-28.

[11] 张伟, 刘芳. 跨境贸易合规风险识别与制裁名单筛查技术研究[J]. 国际贸易问题, 2023(9): 55-68.

[12] Spring Team. Spring Boot 3.3 Reference Documentation[EB/OL]. (2025-05-22)[2026-10-20]. https://docs.spring.io/spring-boot/.

[13] Vue.js Team. Vue 3 Documentation[EB/OL]. (2025-06-10)[2026-10-20]. https://vuejs.org/guide/.

[14] FastAPI. FastAPI Documentation[EB/OL]. (2025-07-15)[2026-10-20]. https://fastapi.tiangolo.com/.

[15] 李明. 基于 Vue 3 与 Spring Boot 的前后端分离系统设计[J]. 电脑知识与技术, 2024, 20(12): 45-48.

附录

附录收录系统实现中的关键代码片段，供评阅参考。

附录 A　网关 JWT 鉴权与租户头注入

```
String token = extractToken(exchange);
if (!StringUtils.hasText(token)) {
    return reject(exchange, 401, "未登录或 Token 已过期");
}
Claims claims = jwtParser.parse(token);
if (claims == null) {
    return reject(exchange, 401, "未登录或 Token 已过期");
}
Long tenantId = jwtParser.tenantIdOf(claims);
String tenantCode = jwtParser.tenantCodeOf(claims);
var mutated = exchange.getRequest().mutate();
if (StringUtils.hasText(internalSecret)) {
    mutated.header(SystemConstants.HEADER_INTERNAL_SECRET, internalSecret);
}
if (tenantId != null) {
                                mutated.header(SystemConstants.HEADER_TENANT_ID,
String.valueOf(tenantId));
}
if (tenantCode != null) {
    mutated.header(SystemConstants.HEADER_TENANT_CODE, tenantCode);
}
return chain.filter(exchange.mutate().request(mutated.build()).build());
```

附录 B　供应商智能匹配评分

```
# 七维加权评分：品类、资质、产品、地域、行业、完整度、出口经验
total = round(
    category_score * 0.30     # 品类覆盖率
    + cert_score * 0.25       # 资质匹配率
    + product_score * 0.15    # 产品关键词
    + region_score * 0.10     # 地域匹配
    + industry_score * 0.05   # 行业匹配
    + completeness * 0.10     # 画像完整度
    + export_level * 0.05,    # 出口经验
    1,
)
# 低于最低分阈值的候选直接过滤
if total < MIN_SCORE:
```

```
   continue
```

附录 C　用户表建表语句

```
CREATE TABLE `user` (
  `id` bigint NOT NULL COMMENT '用户编号',
  `tenant_id` bigint NOT NULL COMMENT '所属租户',
  `username` varchar(64) NOT NULL COMMENT '登录用户名',
  `password_hash` varchar(100) NOT NULL COMMENT 'BCrypt 密码密文',
  `real_name` varchar(64) DEFAULT NULL COMMENT '真实姓名',
  `email` varchar(128) DEFAULT NULL COMMENT '电子邮箱',
  `phone` varchar(32) DEFAULT NULL COMMENT '手机号码',
  `status` varchar(20) NOT NULL DEFAULT 'active' COMMENT '账号状态',
  `last_login_time` datetime DEFAULT NULL COMMENT '最后登录时间',
  `is_deleted` tinyint NOT NULL DEFAULT 0 COMMENT '逻辑删除',
  `create_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `update_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP
     ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_user_tenant_username` (`tenant_id`, `username`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='系统用户表';
```

致谢

在本次课程设计过程中，指导教师在选题、需求分析、系统设计和论文撰写等方面给予了悉心指导，提出了许多宝贵意见，在此表示衷心的感谢。同时感谢项目组同学在需求讨论、接口联调和测试过程中给予的帮助，感谢开源社区提供的 Spring Boot、Vue、MyBatis-Plus、FastAPI、Milvus 等优秀项目，为本系统的实现提供了坚实基础。

由于本人知识水平和开发时间有限，系统中仍存在不足之处，恳请各位老师批评指正。